

Fog Carport

Datamatiker 2. Semester eksamen

Udviklere:

Morten Jensen

[E-MAIL UDELAGT]

<https://github.com/Mortenjenne>

Daniel Hangaard

[E-MAIL UDELAGT]

<https://github.com/DHangaard>

Antal anslag inklusive mellemrum

71,925

17. december 2025

Links til GitHub repository, video demo samt deployet website kan ses efter indholdsfortegnelsen.

Indholdsfortegnelse

Links	4
Indledning	5
Baggrund.....	5
Virksomheden/Forretningsforståelse.....	6
Interessentanalyse (Beskrivelse).....	6
Risikoanalyse (Beskrivelse)	6
Interessentanalyse (Model)	8
Risikoanalyse (Uddrag)	9
Teknologi valg	10
Krav	10
Opsamling på krav	14
Arbejdsgange der skal IT-støttes.....	14
AS-IS Aktivitetsdiagram (Nuværende Arbejdsgang).....	14
TO-BE Aktivitetsdiagram (Foreslået Arbejdsgang)	15
Forskel i Arbejdsgange	15
AS-IS (Diagram) – 1 af 3	16
AS-IS (Diagram) – 2 af 3	17
AS-IS (Diagram) – 3 af 3	18
TO-BE (Diagram) – 1 af 3	19
TO-BE (Diagram) – 2 af 3	20
TO-BE (Diagram) – 3 af 3	21
User stories	22
Domæne model og ER-Diagram	23
Domænemodel (Beskrivelse)	24
Entity Relationship Diagram (Database Implementering)	24
Domænemodel (Diagram)	26
Entity Relationship Diagram (Diagram)	27
Navigationsdiagram & Mockups	28
Navigationsdiagram for Kunder	28
Navigationsdiagram for Sælgere	29
Navigationsdiagram – Kunde model	31

Navigationsdiagram – Sælger (ADMIN) model	32
Valg af arkitektur	33
MVC med Servicelag.....	33
Særlige forhold	34
Session, request og application-variabler.....	34
Exception-håndtering	35
Inputvalidering	35
Sikkerhed ved login	36
Brugertyper (roller).....	36
Styklisteberegner (BomService)	36
Tegning (SVG-generering)	37
AppProperties – centraliseret konfiguration	37
Email-integration (SendGrid)	38
Sammenfatning.....	38
Kodeeksempler.....	38
Kunde-flow – Fra tilbud til ordre	38
Opdatering af materialer i en stykliste	41
Opdatering af materialer i databasen	42
Beregning af stykliste – teknisk flow	44
Status på implementering.....	48
Kvalitetssikring (test)	49
Automatiserede tests.....	49
User Acceptance tests	50
Proces.....	52
Arbejdsprocessen faktuel	52
Projektet i faser	52
Kanban.....	52
Vejledning	53
Kommunikation	53
Arbejdsprocessen reflekteret	54
Scrum master	54
Møder	54

User stories	55
Tidsestimering	55
Rytme i processen.....	56
Bilag.....	57
Bilag 1: Citater fra kundebesøg (Video interview med Martin)	57
Bilag 2: Risikoanalyse	60
Bilag 3: Domain Model Script - Relationer.....	65
Bilag 4: User stories	66
Bilag 5: Kunde e-mails.....	74
Bekræftelse på forespørgsel.....	74
Tilbud.....	75
Ordrebekræftelse.....	76
Bilag 6: Detaljeret plan	78

Links

Link til GitHub repository:

<https://github.com/DHangaard/fog-carport>

Link til videopræsentation:

<https://youtu.be/LoZwtGzrctM>

Link til mockups (Figma)

[UDELADT]

Link til deployeret version:

[IKKE LÆNGERE TILGÆNGELIG]

Login info

Bruger	E-mail	Password
KUNDE	[UDELADT]	[UDELADT]
SÆLGER (ADMIN)	[UDELADT]	[UDELADT]

Indledning

Denne rapport er udarbejdet som en del af Datamatikeruddannelsens 2. semester eksamensprojekt og tager udgangspunkt i en konkret opgave for virksomheden Johannes Fog A/S. Projektet handler om at udvikle et nyt system, der skal bruges i forbindelse med bestilling og håndtering af skræddersyede carporte.

Fog benytter i dag et ældre, internt værktøj til dette formål, men systemet kan ikke længere vedligeholdes og passer ikke til den måde, sælgerne arbejder på. Derfor har virksomheden et ønske om en løsning, hvor kunder kan bestille en carport med egne mål direkte via Fogs hjemmeside, samtidig med at sælgernes arbejdsproces samles i ét samlet flow.

Projektgruppen har valgt at løse opgaven ved at udvikle en prototype baseret på MVC-arkitektur med et servicelag. Prototypen demonstrerer, hvordan et fremtidigt system kan fungere i praksis for både kunder og sælgere, og giver Fog et samlet indblik i struktur, funktionalitet og den digitale brugerrejse.

Baggrund

Johannes Fog A/S er en dansk, fondsejet trælasthandel og byggemarkedskæde med butikker og trælastcentre på Sjælland. Virksomheden er kendt for sin faglige rådgivning, kvalitetsmaterialer og et stærkt fokus på bæredygtighed og ansvarlig drift – værdier, der samlet beskrives som at arbejde “med omtanke”.

Fog leverer løsninger til både private og professionelle kunder, herunder skræddersyede carporte og andre individuelle konstruktioner. Med sin kombination af erfaring, service og langsigtet kvalitet arbejder Fog efter sloganet “*Holder i længden*”, hvilket afspejler virksomhedens tilgang til både produkter, kunderelationer og drift.

Fog har i mange år benyttet et ældre program til udarbejdelse af carporte. I dette system var det ikke muligt for kunder selv at angive mål eller tilpasse en løsning; i stedet skulle de altid kontakte en sælger, som manuelt indtastede oplysningerne på kundens vegne. Programmet har derfor fungeret som et internt arbejdsredskab frem for en løsning, kunden selv kunne anvende.

Det gamle system har samtidig flere praktiske udfordringer. Det er besværligt at arbejde i for sælgerne og kræver, at de skifter mellem forskellige programmer for at kunne gennemføre en beregning, give et tilbud og håndtere materialerne. Mange af de materialer, systemet indeholder, er udgået eller ikke længere relevante, og der er ingen reel mulighed for at opdatere eller vedligeholde data, hvilket gør både udregninger og rådgivning unødigt tidskrævende. Alt dette betyder, at arbejdsprocessen bliver langsom, sårbar og afhængig af sælgerens egne manuelle løsninger.

På et overordnet plan ønsker Fog derfor et nyt program, der gør det muligt for kunder at bestille en carport med skræddersyede mål direkte via virksomhedens hjemmeside som en integreret del af deres digitale løsning. Samtidig skal systemet samle sælgernes workflow ét sted, så de ikke længere behøver at benytte flere systemer for at udføre deres arbejde. Derudover lægger Fog vægt på, at programmet kan vedligeholdes løbende, så materialer og data altid afspejler virksomhedens reelle sortiment og muligheder.

Virksomheden/Forretningsforståelse

Som en del af vores forretningsforståelse udarbejdede vi både en interessentanalyse og en risikoanalyse for projektet. Formålet var at få et overblik over de aktører, der spiller en rolle i forbindelse med Fogs arbejde med skræddersyede carporte, og samtidig identificere de risici, der kunne påvirke vores udviklingsproces.

Interessentanalyse (Beskrivelse)

Selvom vi ikke havde en reel kunde i projektforløbet, udarbejdede vi en interessentanalyse for at skabe et fælles billede af, hvordan Fog som virksomhed er organiseret, og hvilke roller der har betydning for opgaven. Analysen gav os en bedre forståelse af, hvordan virksomheden arbejder – eksempelvis hvilke grupper der har høj indflydelse, hvem der bidrager til arbejdsprocesserne, og hvem der primært skal informeres.

I praksis fyldte interessentanalysen ikke meget i selve udviklingsforløbet, da der ikke var direkte dialog med Fog undervejs. Den har dog fungeret som et nyttigt udgangspunkt for vores forståelse af Fogs interne struktur og sandsynlige forventninger til et system som dette.

(Interessentanalysen er vedhæftet senere i dette afsnit.)

Risikoanalyse (Beskrivelse)

Risikoanalysen blev udarbejdet med udgangspunkt i en virksomhed i drift, hvilket afspejles i formuleringer og problemstillinger, der relaterer sig til en reel udviklingsafdeling. Denne tilgang blev valgt, fordi opgaven forsøger at simulere en professionel virkelighed, og fordi mange af de identificerede risici fortsat er relevante, selv i en mindre projektgruppe som vores.

Vi udvalgte fire risici, som vi vurderede særligt relevante for vores projektforløb, og brugte dem aktivt som pejlemærker i den løbende planlægning og samarbejde.

Mergekonflikt i developer branch

I løbet af projektet oplevede vi enkelte mergekonflikter, især når flere arbejdede parallelt på den samme del af systemet. Risikoanalysen havde gjort os opmærksomme på, at

dette kunne blive en udfordring, og vi valgte derfor kun at merge via pull requests. Det viste sig at være en effektiv strategi, da konflikter kunne løses i den enkelte feature-branch, før de nåede developer-branchen. Når konflikter opstod, fulgte vi planen fra analysen, og håndteringen blev derfor relativt uproblematisk.

Misforståelse af kundekrav i udviklingsteamet

Selvom vi ikke havde direkte adgang til en reel kunde, opstod der perioder, hvor vi internt havde forskellige fortolkninger af Fogs krav. Risikoanalysen gjorde os opmærksomme på dette som en sandsynlig faldgrube, og vi indførte derfor fælles gennemgange af krav efter hvert vejledningsmøde. Når der var tvivl, samlede vi spørgsmål til næste møde, så uklarheder kunne blive afklaret tidligt. Det styrkede vores fælles forståelse og gjorde kravgrundlaget mere konsistent. Et forbedringspunkt i et fremtidigt projekt kunne være at udarbejde et mere formelt kravkatalog fra starten.

Sygdom i udviklingsteamet

Der forekom perioder med sygdom, men risikoanalysen havde forberedt os på dette. Vi lagde vægt på tydelig opgavestatus og brugte GitHub Projects aktivt sammen med løbende kommunikation på Discord. Det gjorde det nemt at overtage eller parkere opgaver ved fravær, og udviklingen fortsatte uden store forstyrrelser. Risikoen viste sig derfor lettere at håndtere, end vi først havde forventet.

Overskridelse af tidsplan i udviklingsteamet

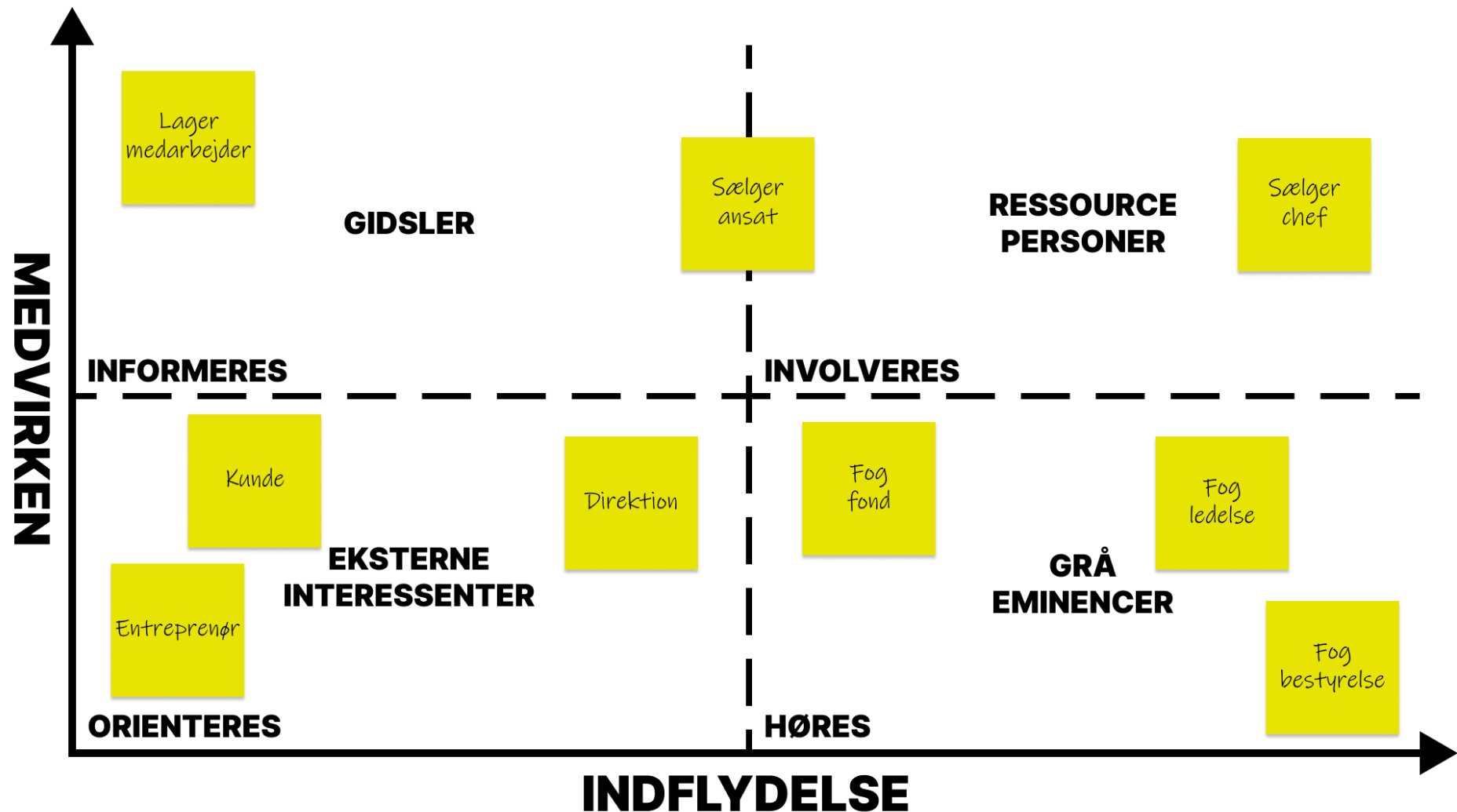
Tidsplanen blev udfordret flere gange, særligt da tegnefunktionen og styklisteberegneren viste sig mere komplekse end forventet. Risikoanalysen pegede på behovet for løbende re-estimering, og dette implementerede vi i vores scrum-møder. Det gjorde det muligt at justere prioriteringer og forventninger undervejs. Selvom vi måtte fravælge enkelte funktioner, gjorde risikoanalysen os bedre forberedte på dette, og vi kunne håndtere tidsudfordringerne mere struktureret. Til en anden gang ville endnu mindre, mere afgrænsede tasks kunne forbedre præcisionen af estimererne yderligere.

Samlet set har risikoanalysen bidraget positivt til vores proces. Den har hjulpet os med at identificere potentielle problemer tidligt, arbejde mere struktureret og reagere hurtigere, når udfordringer opstod. Erfaringerne viser tydeligt, at selv en analyse oprindeligt formuleret til en større organisation kan skabe værdi i et mindre studieprojekt.

Den fulde risikoanalyse kan findes i Bilag 2: Risikoanalyse.

(Den relevante del af risikoanalysen er vedhæftet senere i dette afsnit.)

Interessentanalyse (Model)



Risikoanalyse (Uddrag)

ID	Risiko	Alvor	Sandsynlighed	Risikoniveau	Plan for forebyggelse / afværgning
8	Mergekonflikt i developer branch	Tolereres	Sandsynligt	Høj	Der må udelukkende merges til developer branch i forbindelse med pull requests (med undtagelse af Tech lead). Såfremt der opstår mergekonflikt, løses dette optimalt i featurebranches via Git Pull forud for merge til dev. Ved tvivl, kontakt Tech lead
17	Misforståelse af kunde krav i udviklingsteamet	Tolereres	Muligt	Medium	Scrum master sørger for daglige Scrum møder. Udviklingsteamet gennemgår de daglige tasks med Product owner ved tvivl. Såfremt at Product owner støder på en tydelig misforståelse ved teamet skal denne kontakte relevant personel for at udbedre
19	Sygdom i udviklingsteamet	Tolereres	Sandsynligt	Medium	Hver udvikler skal have en klar plan for overdragelse og status på igangværende opgave i tilfælde af fravær. Scrum master har ansvaret for at uddelegere opgaver i forbindelse med dette
21	Overskridelse af tidsplan i udviklingsteamet	Tolereres	Sandsynligt	Medium	Udviklingsteamet melder frem til Scrum master og sender en ny estimeret tid for færdiggørelse. Scrum master skal ved gentagende overskridelser revurdere estimering af tid til tasks. Ved næste daglige Scrum møde revurderes prioritering af opgaver. Såfremt overskridelsen er kritisk, har Scrum master ansvaret for at vurdere og prioritere opgaver og derefter informere relevant personel

Teknologi valg

Teknologisk Lag	Teknologiområde	Navn	Version
Backend & Core	Primær Sprog	Java	-
	Java Development Kit	JDK	17
	Web Framework	Javalin	6.1.3
	Build Tool	Apache Maven	3.10.1
Data & Test	Database System	PostgreSQL	16.2
	JDBC Driver	PostgreSQL JDBC Driver	42.7.2
	Test Framework	JUnit	5.10.2
Frontend & Presentation	Markup sprog	HTML	5
	Styling sprog	CSS	3
	Scripting sprog	JavaScript	ES6
	Template Engine	Thymeleaf	3.1.2
Infrastructure & Deployment	Container	Docker	28.5.1
	Web Server / Reverse Proxy	Caddy	2.7.6
	Hosting Service / Cloud Provider	Digital Ocean	-
	E-mail Service	Sendgrid	4.10.3
Development Tools	Integrated Development Environment (IDE)	IntelliJ IDEA Ultimate	2025.3
	Version Control	Git via GitHub	-

Krav

Kravene til systemet er udledt på baggrund af et interview med Martin, medarbejder hos Fog, hvor det nuværende system og arbejdsgange blev gennemgået. Interviewet har dannet grundlag for forståelsen af både systemets begrænsninger og de forretningsmæssige hensyn, Fog ønsker at bevare.

Interviewet dækker ikke tekniske løsninger eller user stories, men beskriver behov og udfordringer ud fra den daglige praksis. Kravene i dette afsnit er derfor formuleret i brede, overordnede termer og vil senere i rapporten blive omsat til user stories.

Krav 1

Citat: *“Lige nu kan du bestille en carport. Du kan også gå ind og lave denne forespørgsel til de her specialmål og gør man det, i det øjeblik man trykker på knappen, så lander der en mail herude med de ønsker folk har. Og så har vi vores eget lille program, hvor vi så, lavpraktisk, med de mål folk har tastet ind [...] taster noget bredde og noget længde [...]”*
— Martin Kristensen (Fog), interview 00:04:08

“Den kan du ikke se herinde, der kan jeg gå herind og arbejde i en prisdatabase.”
— Martin Kristensen (Fog), interview 00:16:06

(Svar på spørgsmål om hvor materialeprisen fremgår.)

Udledt krav: Systemet skal konsolideres således det bliver mere brugervenligt for en sælger.

Begrundelse: Martin beskriver en fragmenteret proces, hvor information om specialmål kommer ind via mail og priser skal tilgås via en separat database. For at optimere sælgerens arbejdsgang og reducere fejlriskoen er der et klart behov for at samle funktionaliteten i ét system.

Krav 2

Citat: *“Og så har vi vores eget lille program, hvor vi så, lavpraktisk, med de mål folk har tastet ind [...] taster noget bredde og noget længde [...]”*
— Martin Kristensen (Fog), interview 00:04:08

Udledt krav: Systemet skal automatisere håndteringen af brugerinput og på baggrund heraf, automatisk generere den nødvendige stykliste og tegning.

Begrundelse: Den nuværende proces er "lavpraktisk" og manuel, hvilket er tidskrævende. Systemet skal sikre, at brugerens ønskede mål direkte kan oversættes til en præcis stykliste og tegning uden manuel indtastning, hvilket øger effektiviteten og mindsker den administrative byrde for sælgeren.

Krav 3

Citat: *“Det her er et program fra ’99 [...] Problemet er faktisk, at jeg har et kodeord så jeg kan gå ind og rette priser på det, men ellers kan vi faktisk ikke lave noget. Vi kan ikke rette varer, og vi kan ikke oprette nye varer.”*
— Martin Kristensen (Fog), interview 00:13:21

“Jeg kan ingenting rette i det andet end jeg kan rette priserne. Det var et rent tilfælde at jeg fandt kodeordet.”
— Martin (Fog), interview 00:16:38

“En taglægte i dag hedder ikke 38x57, den hedder 38x73. Det kan vi faktisk ikke ændre i vores program. Jeg har ændret prisen til en 73 pris, så maskinen kommer med den rigtige pris. Og når kunden han får den, så får han jo denne her tegning, men styklisten, den er tastet i vores edb-system. Det er efter den vi pakker efter. Så det giver selvfølgelig nogle udfordringer med nogle hjælpe tekster, men det er vi simpelthen nødt til, for materialerne har simpelthen ændret sig på 20 år.”

— Martin Kristensen (Fog), interview 00:17:13

Udledt krav: Systemet skal understøtte CRUD-operationer (Create, Read, Update, Delete) for hele varekataloget.

Begrundelse: Det nuværende system er statisk og forhindrer Fog i at tilpasse sig markedet og materialernes udvikling. Manglen på mulighed for at oprette nye varer eller opdatere dimensioner gør systemet forældet og giver fejl i kundedokumentationen.

Krav 4

Citat: *“Det frygtelige er, at vi har en anden udgave liggende [af programmet], som har ligget i syv år, men som aldrig kom til at virke rigtigt. Det den skulle kunne, det var at hente varenumrene direkte nede i vores database. Det vil sige, at når der kom en prisopdatering, så fulgte den med hele vejen.”*

— Martin Kristensen (Fog), interview 00:14:05

Udledt krav: Systemet skal være integreret med Fogs centrale database for automatisk at hente opdaterede varenumre, priser og specifikationer.

Begrundelse: Formålet er at sikre datakonsistens. Hvis systemet automatisk henter priser og varedata, elimineres risikoen for at arbejde med forældede oplysninger og behovet for manuel prisopdatering mindskes.

Krav 5

Citat: *“Vi ønsker ikke at dele en stykliste til en kunde der får et tilbud. Han får et tilbud på carporten, og så får han styklisten det øjeblik han køber den.”*

— Martin Kristensen (Fog), interview 00:08:13

“Vi er en ganske almindelig trælast. Alt hvad vi sælger herfra, det er varer taget ude fra hylden. [...] Så ved at der blev leveret en stykliste, så ville vores kompetence være væk og så ville der være fri konkurrence.”

— Martin Kristensen (Fog), interview 00:19:30

(Svar på spørgsmål om hvorfor styklisten ikke bliver sendt til kunden med et tilbud.)

Udledt krav: Systemet skal sikre, at kunden ikke modtager en stykliste sammen med tilbuddet.

Begrundelse: Dette er et ikke-funktionelt forretningskrav for at beskytte Fogs *know-how* og konkurrenceevne. Selvom styklisten skal genereres internt til sælgerens brug (jf. Krav 2), skal systemet på intet tidspunkt inkludere eller gøre styklisten tilgængelig for kunden, før købet er bekræftet (jf. Krav 7).

Krav 6

Citat: *“Den har en kort beskrivelse – det er den beskrivelse der bliver skrevet med ud på tegningen, når vi sender kunden. Jamen hvad er det? Det er en carport der er det mål, den har et redskabsrum på der er det mål [...] Det er sådan basis overordnet. Det gør vi selvfølgelig, fordi vi ønsker ikke at dele en stykliste til en kunde der får et tilbud. Han får et tilbud på carporten, og så får han styklisten det øjeblik han køber den.”*

— Martin Kristensen (Fog), interview 00:08:13

Udledt krav: Et tilbud til kunden skal som minimum indeholde en overordnet beskrivelse af produktet, de valgte mål, en tegning og den samlede pris.

Begrundelse: Kunden har behov for at vide, hvad de køber (mål og udformning), samt hvad det koster (pris). Den overordnede beskrivelse og tegningen er tilstrækkelig dokumentation i tilbudsfasen.

Krav 7

Citat: *“Han får et tilbud på carporten, og så får han styklisten det øjeblik han køber den.”*

— Martin Kristensen (Fog), interview 00:08:13

Udledt krav: Når en ordre er bekræftet, skal systemet automatisk levere styklisten til kunden (sammen med tilbudsdokumentationen).

Begrundelse: Efter købet skal kunden have den nødvendige dokumentation for at kunne bygge produktet eller for at Fog kan pakke varerne.

Krav 8

Citat: *“Når vi laver de her [carporte med skræddersyede mål], så tager vi jo dialogen med kunden over disken, og så aftaler vi hvad er det der skal ske, hvad er det du godt kan tænke dig.”*

— Martin Kristensen (Fog), interview 00:09:45

“Vi kan egentlig godt lide at der er en sælger indover”

— Martin (Fog), interview 00:21:20

Udledt krav: Systemet skal designes til at understøtte sælgerens rådgivende og personlige rolle i salgsprocessen, f.eks. ved at gemme kundeforespørgsler til opfølgning.

Begrundelse: Selvom systemet skal automatisere beregninger, er den personlige kontakt en vigtig forretningsstrategi for Fog. Systemet skal derfor ikke udelukke sælgeren, men tværtimod give sælgeren et værktøj, der frigør tid til den personlige dialog.

Opsamling på krav

De udledte krav kan sammenfattes således:

- **Krav 1:** Systemet skal konsolideres således det bliver mere brugervenligt for en sælger.
- **Krav 2:** Systemet skal automatisere håndteringen af brugerinput og på baggrund heraf, automatisk generere den nødvendige stykliste og tegning.
- **Krav 3:** Systemet skal understøtte CRUD-operationer (Create, Read, Update, Delete) for hele varekataloget.
- **Krav 4:** Systemet skal være integreret med Fogs centrale database for automatisk at hente opdaterede varenumre, priser og specifikationer.
- **Krav 5:** Systemet skal sikre, at kunden ikke modtager en stykliste sammen med tilbuddet.
- **Krav 6:** Et tilbud til kunden skal som minimum indeholde en overordnet beskrivelse af produktet, de valgte mål, en tegning og den samlede pris.
- **Krav 7:** Når en ordre er bekræftet, skal systemet automatisk levere styklisten til kunden (sammen med tilbudsdokumentationen).
- **Krav 8:** Systemet skal designes til at understøtte sælgerens rådgivende og personlige rolle i salgsprocessen, f.eks. ved at gemme kundeforespørgsler til opfølgning.

Disse krav danner grundlaget for de user stories der fremgår senere i rapporten.

De fulde citater, som danner grundlag for kravene i dette kapitel, kan findes i Bilag 1: Citater fra kundebesøg (Video interview med Martin).

Arbejdsgange der skal IT-støttes

AS-IS Aktivitetsdiagram (Nuværende Arbejdsgang)

AS-IS-modellen beskriver den manuelle og fragmenterede arbejdsgang observeret i interviewet, der foregår på tværs af flere uafhængige systemer:

- **Fragmenteret IT-støtte:** Forespørgslen lander i Fog Mail System, hvorefter sælgeren manuelt taster input ind i Carport Systemet for at beregne styklisten og generere tegningen.
- **Manuel Prisberegning:** Modellen viser, at ukorrekt prissætning kræver opdatering i Lager Systemet, hvilket understreger manglende integration.
- **Administrativ byrde:** Sælgeren bruger væsentlig tid på at håndtere forespørgslen, kontakte kunden og bestemme dækningsgrad (rabat), før tilbuddet kan genereres.

AS-IS-modellen understreger den manglende integration mellem Fogs systemer, hvilket resulterer i manuelle trin og dobbeltarbejde for sælgeren.

(AS-IS-modellen er vedhæftet senere i dette afsnit.)

TO-BE Aktivitetsdiagram (Foreslået Arbejdsgang)

TO-BE-modellen repræsenterer den optimerede arbejdsgang. Den centrale forskel fra AS-IS er konsolideringen af funktionalitet i én svømmebane: Fog Web Applikation.

Det nye system opfylder behovet for øget brugervenlighed og automatisering:

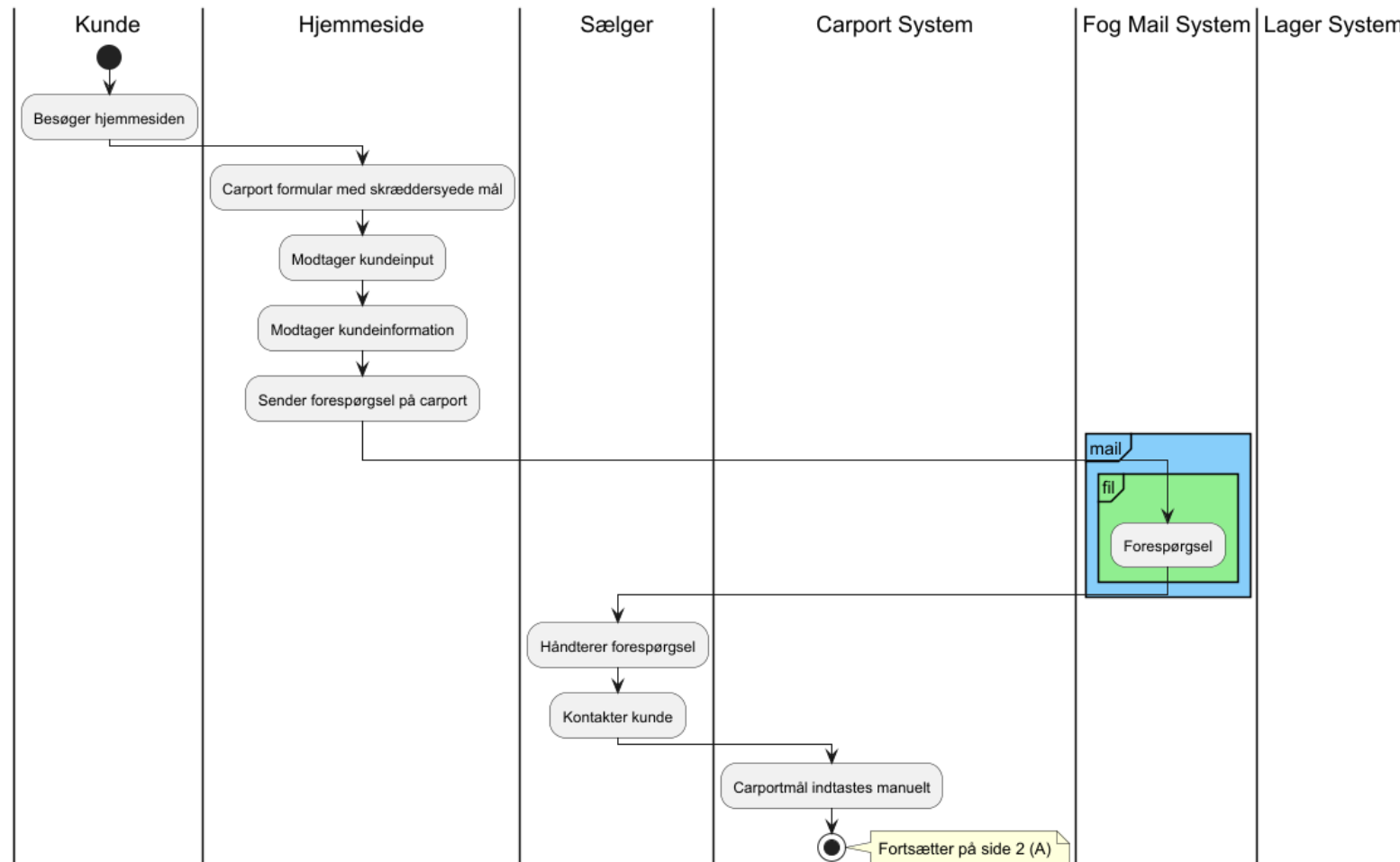
- **Centraliseret Systemstøtte:** Efter kundens forespørgsel overtager Fog Web Applikationen processen. Tunge processer (Beregn carport, Generer stykliste, Generer tegning og Generer tilbud) foregår automatisk og integreret.
- **Reduceret Manuel Indtastning:** Sælgerens rolle flyttes fra systemkoordinering til udelukkende at håndtere og vurdere tilbuddet og justere dækningsgrad/rabat.
- **Styring af Dokumentation:** Kunden modtager en e-mail notifikation om, at informationen er tilgængelig. Tilbuddet indeholder Tilbud og Tegning, mens ordrebekræftelsen specifikt inkluderer Stykliste og Tegning. Dette sikrer, at styklisten ikke inkluderes i et tilbud.

(TO-BE-modellen er vedhæftet senere i dette afsnit.)

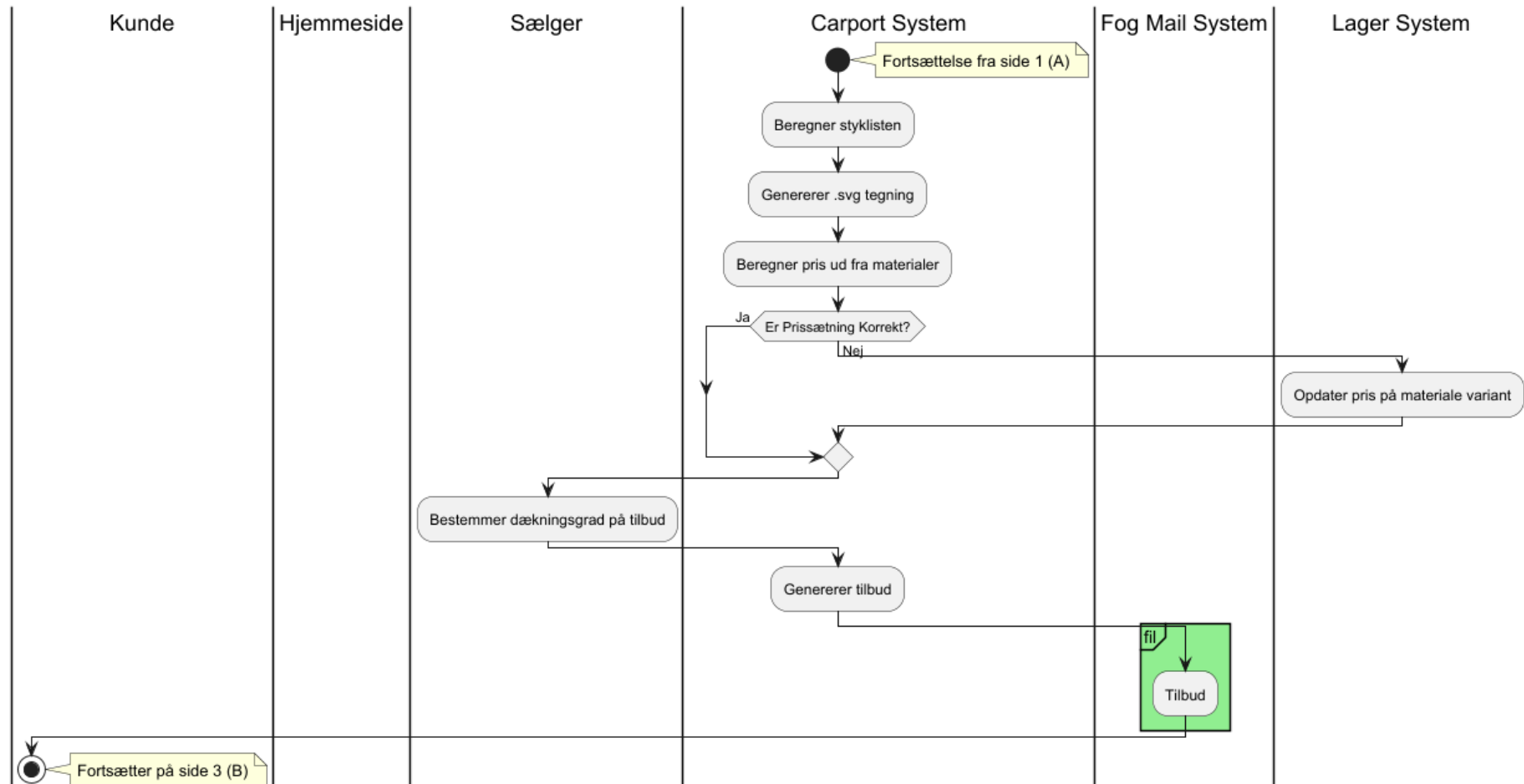
Forskel i Arbejdsgange

Forskellen mellem AS-IS og TO-BE ligger primært i automatiseringen og konsolideringen. AS-IS viser et komplekst samspil mellem fire systemer, mens TO-BE samler central forretningslogik i Fog Web Applikationens svømmebane. Overgangen omdanner den manuelle og ineffektive proces til en strømlinet digital proces.

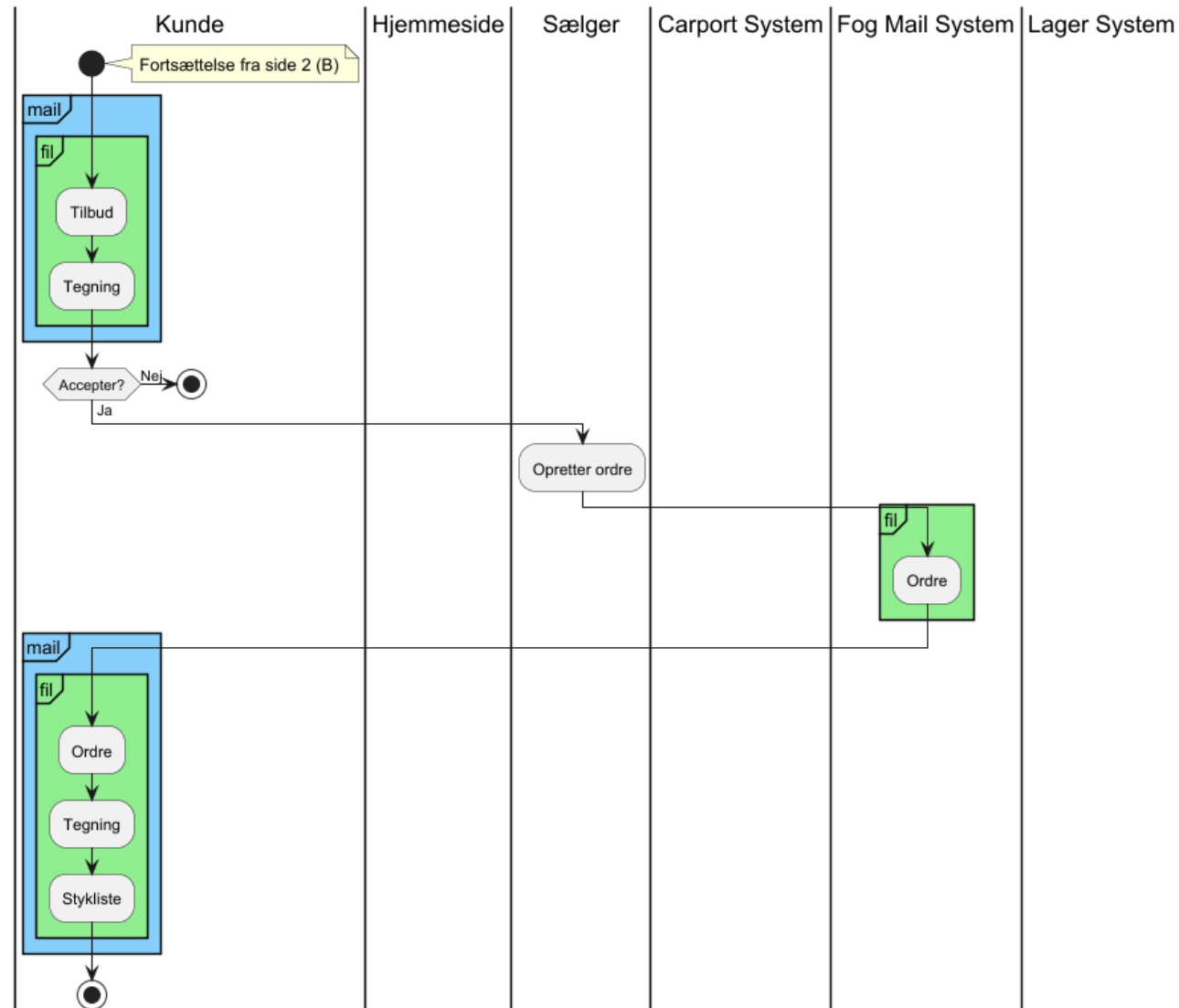
AS-IS (Diagram) – 1 af 3



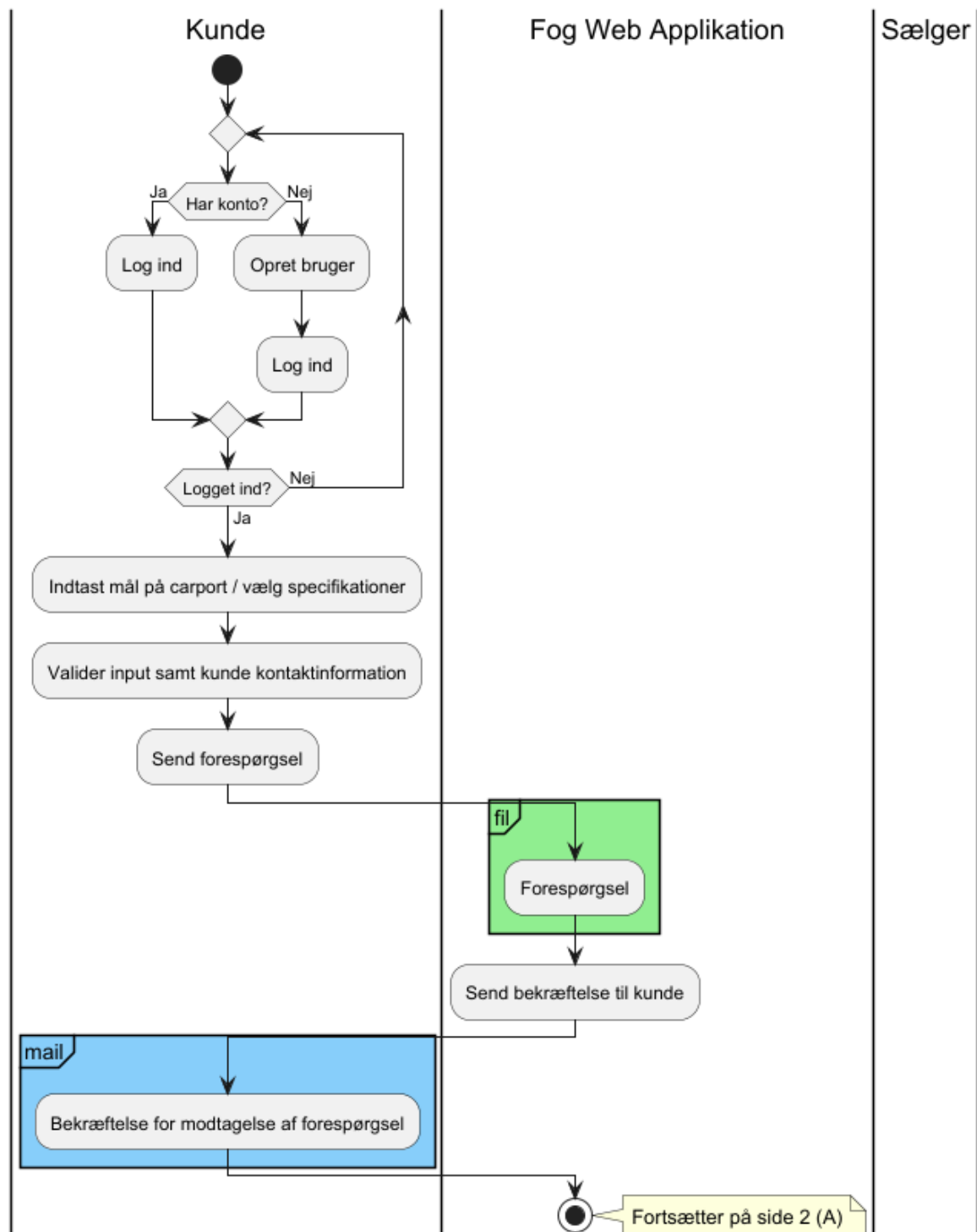
AS-IS (Diagram) – 2 af 3



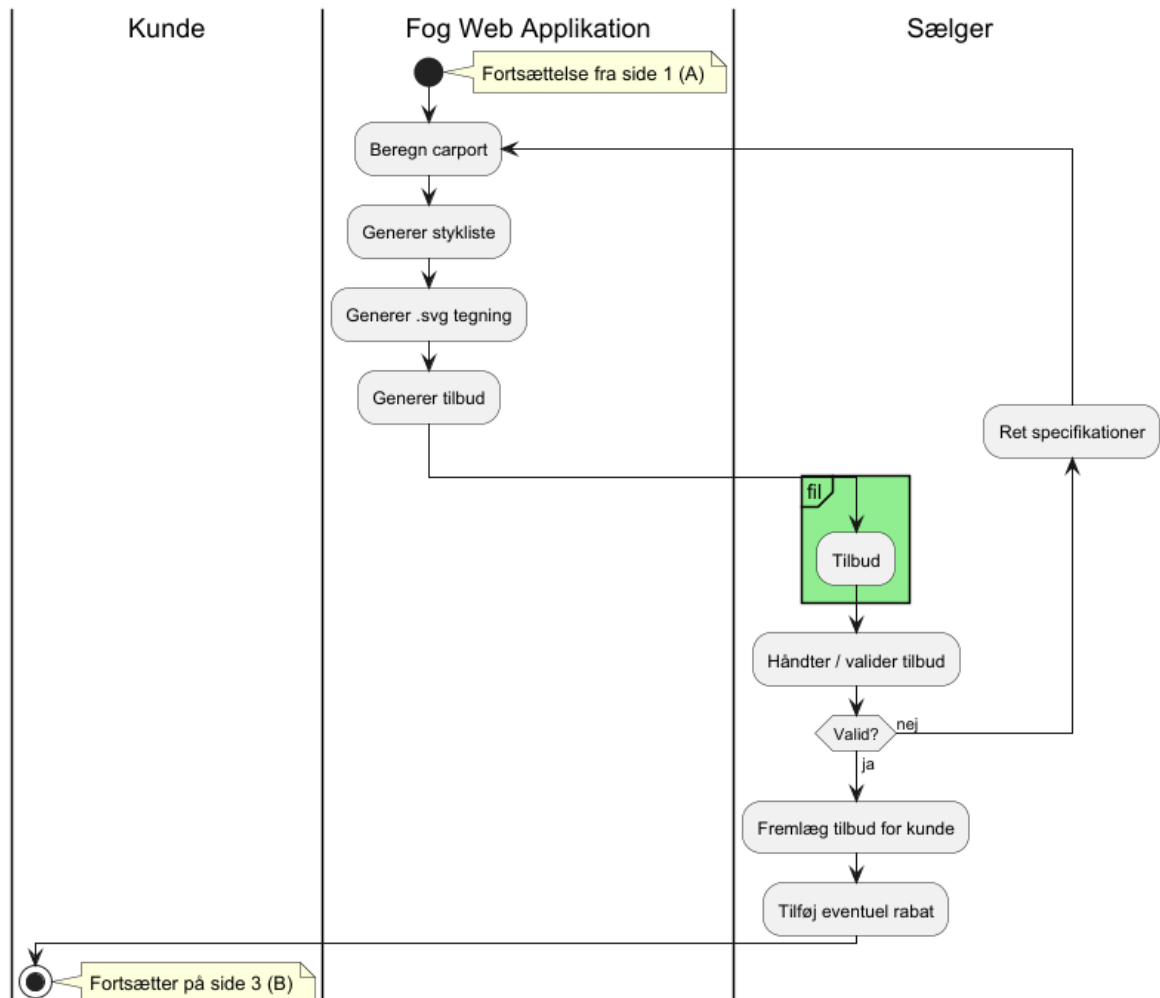
AS-IS (Diagram) – 3 af 3



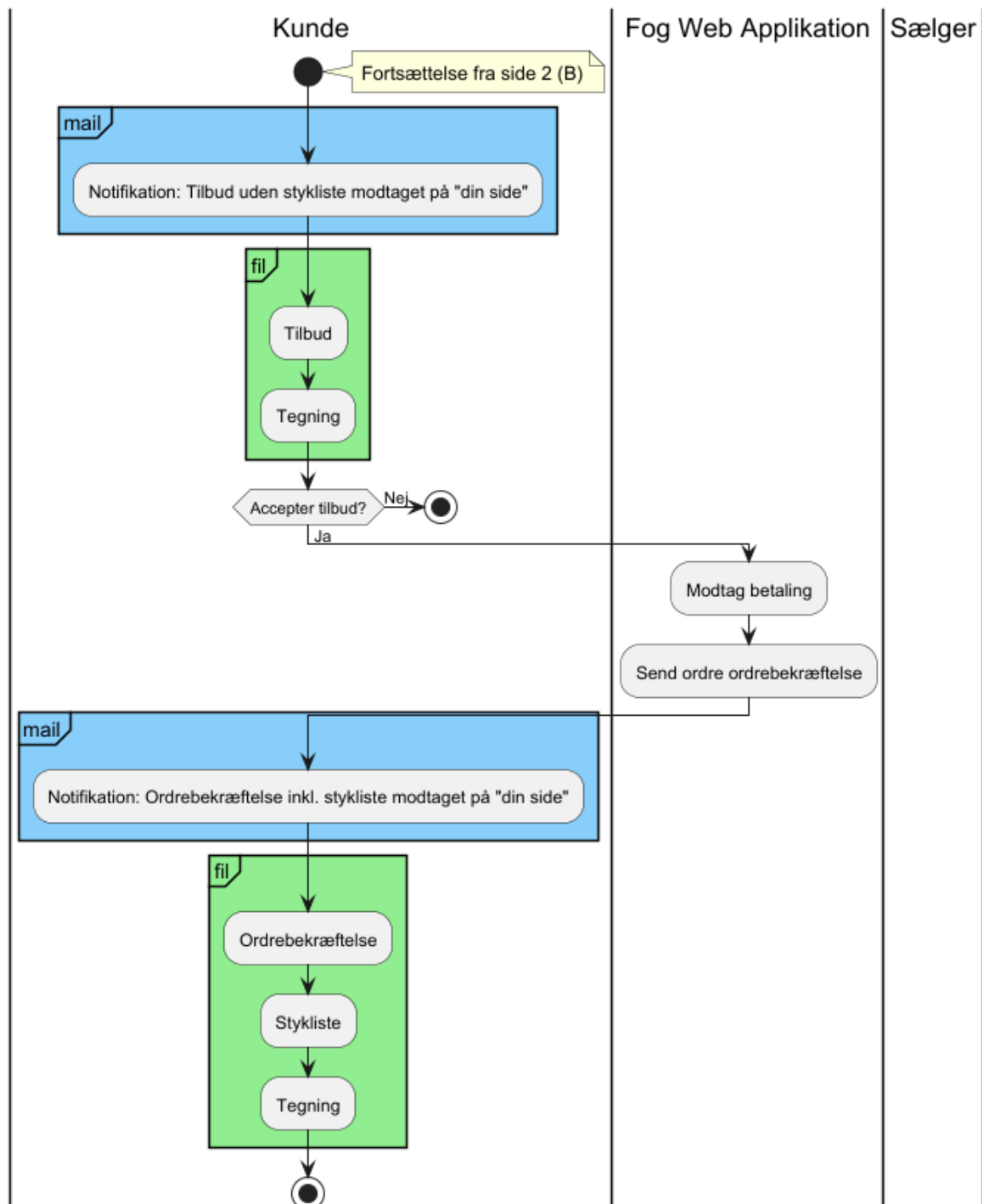
TO-BE (Diagram) – 1 af 3



TO-BE (Diagram) – 2 af 3



TO-BE (Diagram) – 3 af 3



User stories

User stories i projektet er udarbejdet på baggrund af de identificerede krav og ikke i direkte samarbejde med kunden. I dette kapitel præsenteres et udvalgt udsnit af user stories (6, 7, 10, 16 og 22), som alle omhandler central og forretningskritisk funktionalitet beskrevet i kravmaterialet. Disse user stories er valgt, da de danner grundlag for de efterfølgende kodeeksempler og anvendes til at illustrere konkrete løsninger på centrale problemstillinger i systemet.

US-6

Rolle: Kunde

Som kunde ønsker jeg at kunne se mit tilbud på "Min side" under tilbud, når jeg har modtaget et nyt tilbud, så jeg kan se tilbuddet og en tegning af carporten.

Acceptkriterier:

Givet at en sælger har oprettet et tilbud til mig, når jeg logger ind eller er logget ind, og at jeg er på "Min side" når jeg åbner tilbuddet, kan jeg se en kort beskrivelse af carport, tegning og pris.

Estimat: L

Denne user story dækker det første skridt i kunde-flowet, hvor kunden kan se et tilbud, som en sælger har oprettet. På dette tidspunkt kan kunden se en beskrivelse af carporten, en tegning og prisen, men ikke styklisten.

US-7

Rolle: Kunde

Som kunde ønsker jeg at acceptere et tilbud, så jeg kan få styklisten og min ønskede carport.

Acceptkriterier:

Givet at jeg har modtaget et tilbud, når jeg klikker "Accepter" og gennemfører betalingen. Så registreres betalingen, og jeg kan se styklisten og modtage min carport.

Estimat: L

Når kunden accepterer tilbuddet og gennemfører betalingen, skifter ordren status. Dette trin fungerer som overgangen mellem tilbud og egentlig ordre.

US-10

Rolle: Kunde

Som kunde ønsker jeg at kunne se mine betalte ordrer med tilhørende stykliste, tegning og pris, så jeg nemt kan få overblik over mine købte carporte.

Acceptkriterier:

Givet at jeg er logget ind, Når jeg åbner “Min side” og vælger en ordre fra ordrelisten, så vises en oversigt over mine tidligere betalte ordrer.

Når jeg klikker på en specifik ordre, så vises ordredetaljer, stykliste, tegning og samlet pris.

Estimat: M

Efter betalingen kan kunden nu se sine tidligere betalte ordrer. Her vises både ordredetaljer, pris, tegning og nu også styklisten. Dermed fuldendes kunde-flowet.

US-16

Rolle: Sælger

Som sælger ønsker jeg at redigere webapplikationen forslag til stykliste, så jeg kan kontrollere og tilpasse konstruktionen for nøjagtighed.

Acceptkriterier:

Givet at jeg arbejder på et tilbud, når jeg ændrer materiale-mængder, så opdateres stykliste og totalpris automatisk.

Estimat: M

User storyen er udledt af Krav 2 og Krav 8, hvor automatisering kombineres med sælgerens rådgivende rolle.

US-22

Rolle: Sælger

Som sælger ønsker jeg at opdatere materialer i databasen, så priser og beskrivelser altid er korrekte.

Acceptkriterier:

Givet at jeg har adgang til materialebasen, når jeg redigerer et materiale og klikker “Opdater”, så opdateres oplysningerne i webapplikationen.

Estimat: M

Denne user story understøtter fulde CRUD-operationer på materialer og opfylder Krav 3. Sælgere kan oprette, opdatere og slette materialer samt sikre, at priser og beskrivelser altid er korrekte i databasen.

(samtlige user stories, som er udarbejdet i forbindelse med projektet, kan findes i Bilag 4: User stories)

Domæne model og ER-Diagram

Domænemodellen og det efterfølgende ER-diagram udgør det fundamentale grundlag for webapplikationens datastruktur. Domænemodellen fokuserer på de konceptuelle klasser og deres relationer i den virkelige verden, uafhængigt af den tekniske

implementering. ER-diagrammet omsætter disse koncepter til databasestrukturer, tabeller og nøjagtige relationer.

Domænemodel (Beskrivelse)

Domænemodellen beskriver de centrale begreber og sammenhænge i Fog Carports forretningsdomæne og danner grundlag for den efterfølgende database- og systemstruktur. Modellen er uafhængig af teknisk implementering og fokuserer på at afspejle virkelighedens aktører, objekter og processer.

Modellen tager udgangspunkt i salgsprocessen for skræddersyede carporte og omfatter blandt andet kunder, sælgere, forespørgsler, tilbud og ordrer. Forløbet er modelleret som en progression fra forespørgsel (Inquiry) til tilbud (Offer) og videre til en ordre (Order), hvilket afspejler Fog's arbejdsgang fra første kundekontakt til gennemført salg.

Brugere er abstraheret gennem en fælles Account-klasse, hvorfra Customer (kunde) og Seller (sælger) nedarver fælles egenskaber, hvilket understøtter en klar adskillelse af roller. Carporten udgør et centralt domæneobjekt og beskrives gennem mål, eventuelt redskabsrum samt tilhørende materialer, som indgår i en styklister (BillOfMaterials).

Domænemodellen danner grundlag for det efterfølgende ER-diagram, hvor de beskrevne begreber omsættes til en egentlig databasestruktur.

(Domænemodellen er vedhæftet senere i dette afsnit.)

Entity Relationship Diagram (Database Implementering)

ER-diagrammet beskriver den databasestruktur, der er udledt af domænemodellen, og består af otte tabeller, som understøtter systemets centrale funktionalitet.

Carport og ordre-relationer

En ordre refererer altid til præcis én carport-konfiguration via `carport_id`.

Databasestrukturen tillader teknisk set, at flere ordrer kan referere til samme carport, hvilket giver fleksibilitet i systemet. I praksis oprettes dog typisk en ny carport for hver ordre, hvilket sikrer, at ordrer har deres egne konfigurationer, der kan ændres uafhængigt.

Relationen mellem carport og shed er modelleret som 1:0..1, da et skur er en valgfri del af en carport. Disse komponenter er adskilt i separate tabeller frem for at samle alt i én carport-tabel. Dette valg skyldes, at:

- Skuret er en valgfri komponent, som ikke alle carporte har
- En samling ville medføre nullable attributter (shed-specifikke felter), der ikke er relevante for carporte uden skur
- En samling ville bryde 3. normalform ved at introducere værdier der ikke er afhængige af den primære nøgle

Bevidste normalformsbrud

Der er foretaget et bevidst brud på 3. normalform i tabellen orders, hvor beregnede værdier som `cost_price` og `coverage_percentage` gemmes direkte. Dette sikrer, at historiske priser og beregninger bevares uændret, selvom materialepriser eller forretningslogik ændres over tid. Dette er essentielt for at kunne dokumentere præcis, hvad kunden blev tilbudt på et givent tidspunkt.

Mange-til-mange relation

Relationen mellem ordrer og materialer er modelleret som en mange-til-mange-relation via koblingstabellen `material_line`, som indeholder både reference til materialevariant og den anvendte mængde. Dette muliggør, at:

- En ordre kan indeholde mange forskellige materialer
- Et materiale (materialevariant) kan indgå i mange forskellige ordrer
- Den konkrete mængde gemmes som en attribut på relationen

Brugerroller og dobbeltrelationer

Tabellen `users` har en central rolle i systemet med to forskellige relationer til `orders`: som `customer_id` (den der afgiver ordren) og som `seller_id` (den medarbejder der behandler ordren). Dette design understøtter, at:

- En bruger kan have rollen `CUSTOMER` eller `SALESREP`
- Sælgere kan tildeles til at håndtere ordrer fra kunder
- `seller_id` er nullable, da ordrer oprettes af kunder uden en tildelt sælger

Dette giver fleksibilitet til at spore både kundens og den ansvarlige medarbejders relation til ordren

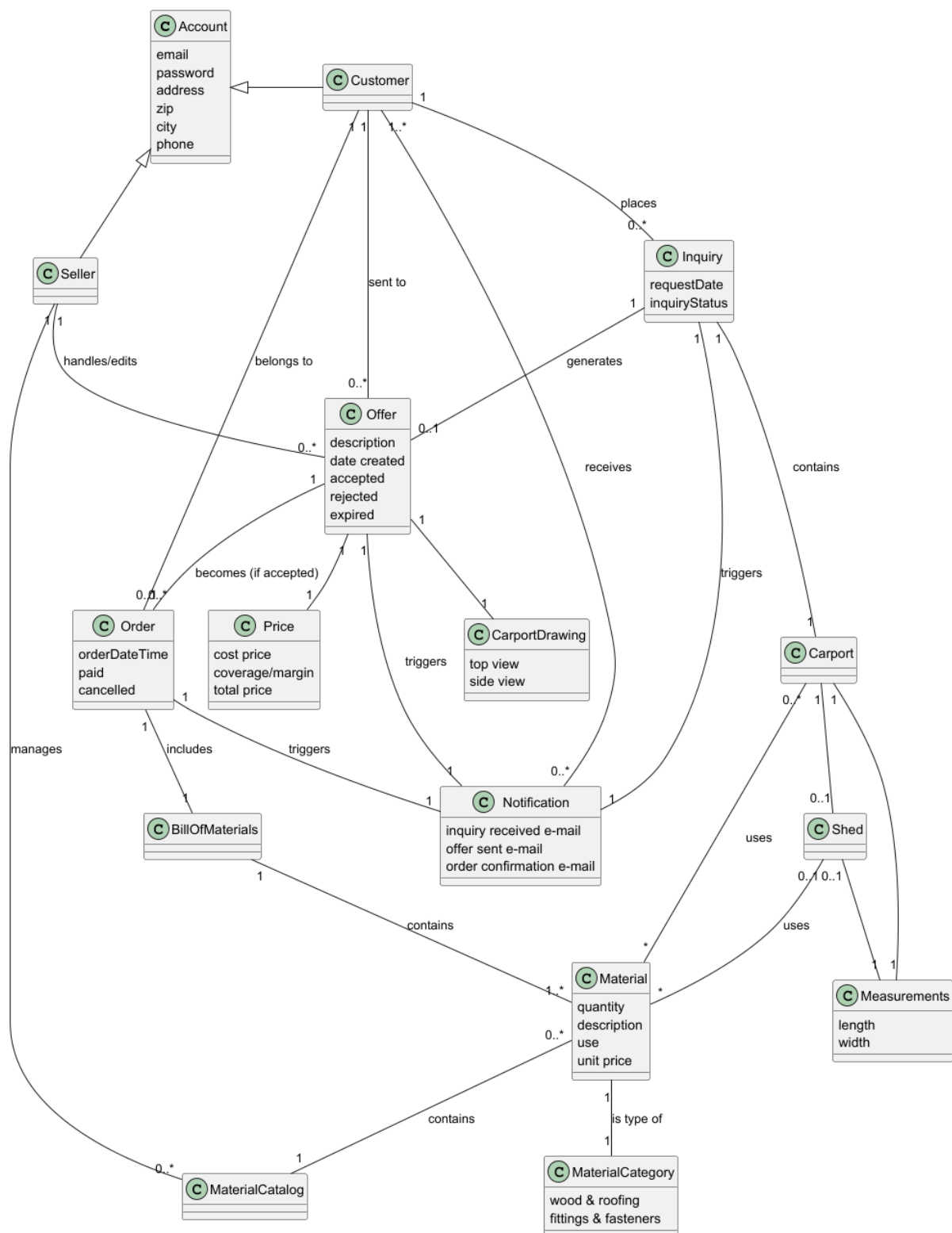
Primærnøgler og navngivning

Alle tabeller anvender automatisk genererede ID'er (`SERIAL`) som primærnøgle, med undtagelse af `zip_code`, hvor postnummeret fungerer som en naturlig primærnøgle. Dette er valgt, da postnumre er standardiserede, unikke identifikatorer i den danske kontekst.

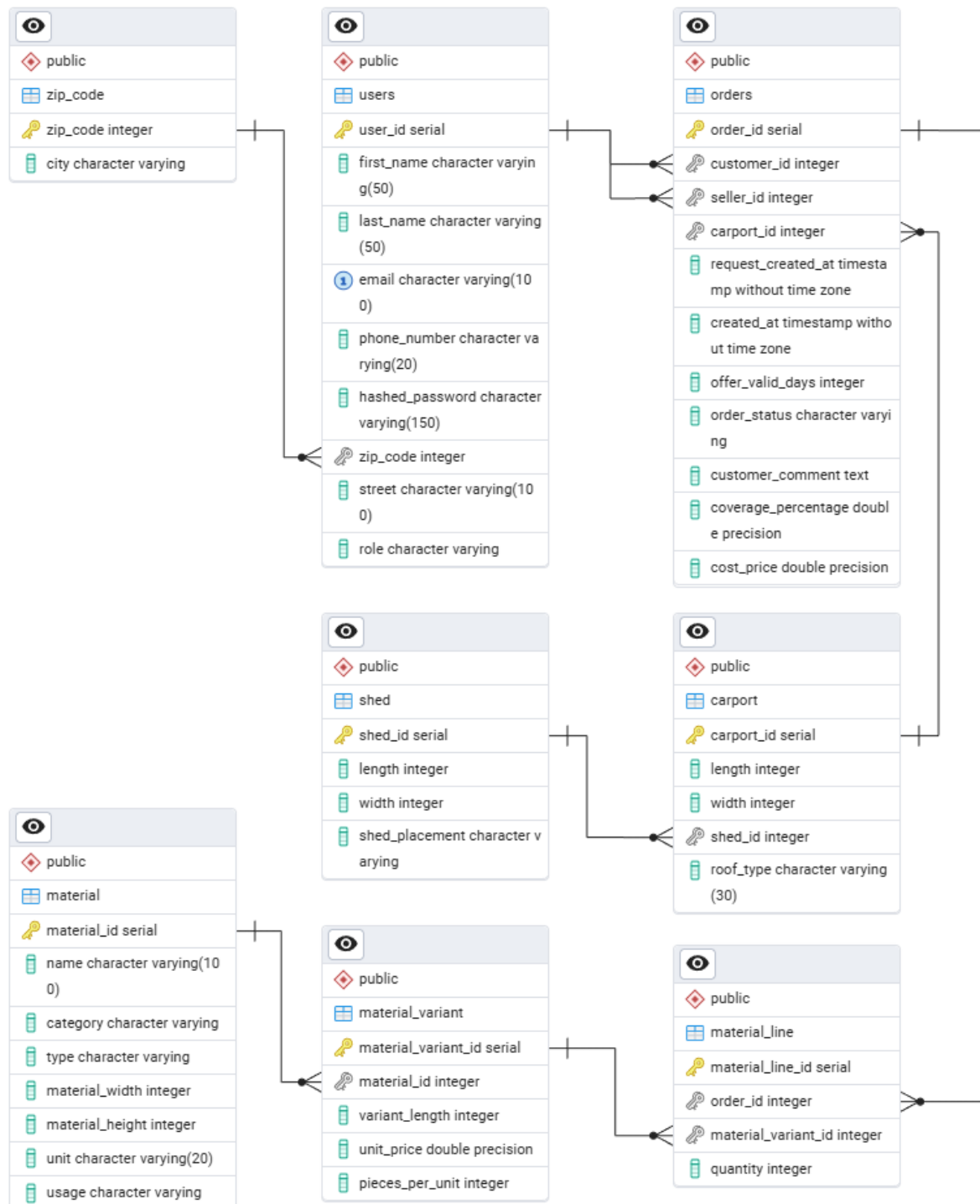
Tabellerne `users` og `orders` er navngivet i pluralis for at undgå konflikt med reservede SQL-nøgleord.

(ERD er vedhæftet senere i dette afsnit.)

Domænemodel (Diagram)



Entity Relationship Diagram (Diagram)



Navigationsdiagram & Mockups

Dette kapitel præsenterer webapplikationens struktur gennem to UML State Machine-diagrammer. Diagrammerne fungerer som et overordnet overblik over de enkelte sider og de ruter (Routes), der forbinder dem, hvilket sikrer et klart billede af systemets navigationslogik. Systemet er opdelt efter brugerroller for at differentiere adgang og funktionalitet, hvilket er afgørende for at opfylde de fastlagte forretningskrav om både kundeservice og intern effektivisering.

Hver tilstand (boks) i diagrammerne indeholder to informationslag:

1. **Top:** Titel (f.eks. ChooseCarport).
2. **Bund:** Den tilhørende Route (f.eks. /carporte) efterfulgt af navnet på den anvendte Thymeleaf skabelon (f.eks. build-carport.html).

(Begge navigationsdiagrammer er vedhæftet senere i dette afsnit.)

Navigationsdiagram for Kunder

Diagrammet illustrerer kundens flow, fra den første interaktion som gæst til den endelige ordrehåndtering. Adgangskontrol er indbygget, så de fleste handlinger kræver login. Denne adgangskontrol er et designvalg i prototypen for at fremvise flest mulige features – såsom opfølgning og tilgang af ordre via ”Min side”. I et færdigt produkt vil en kunde også kunne sende en forespørgsel på en carport uden login.

Flow for Gæst og Login

- Alle brugere starter på forsiden (Index /index.html).
- Fra Index kan en gæst vælge at se carport-muligheder (ChooseCarport /carporte/build-carport.html).
- For at påbegynde carport-formularen (CarportFormular), skal kunden logge ind (Login /login.html).
- Har kunden ingen konto, kan en ny oprettes via siden (CreateUser /create-user.html), hvorefter brugeren kan logge ind.

Flow for Udfyldelse af Forespørgsel

- Efter succesfuldt login udfylder kunden carport-mål og oplysninger på (CarportFormular /carport-formular.html).
- Kunden sendes derefter til (ContactFormular /request-offer-contact.html) for at bekræfte / indtaste kontaktinformation.

- Forespørgslen bekræftes på (RequestConfirmation /request-confirmation.html) og sendes herefter til sælgeren.
- Kunden kan følge forespørgslens status på (MyPage /my-page.html)

Flow for Tilbud og Ordre

- Efter en sælger har udarbejdet et tilbud, navigerer kunden til (MyPage /my-page.html) for at se modtagne tilbud.
- Fra MyPage kan kunden tilgå tilbudsdetaljerne (CarportOfferDetail /customer-offer-details/{id}) for at acceptere eller afvise tilbuddet.
- Accepteres tilbuddet, konverteres det til en ordre. Fra MyPage kan kunden tilgå ordredetaljerne (CarportOrderDetail /carport/details/view/{id}) under betalte ordrer.
- Fra både tilbud og ordre kan kunden åbne tegningen i et nyt vindue (CarportDrawing /show-carport-drawing/{id}).

Navigationsdiagram for Sælgere

Sælgerens navigationsdiagram er designet til at understøtte den administrative arbejdsgang (TO-BE), hvor alle systemfunktioner er samlet i ét. Alle states i dette diagram er kun tilgængelige for brugere med sælgerrettigheder (kræver login).

Hovedområder fra Index

Når en sælger er logget ind, er der direkte adgang fra forsiden (Index /index.html) til de fire centrale administrationsområder:

- **Forespørgsler** (CarportRequests /admin-request.html)
- **Tilbud** (CarportOffers /admin-offers.html)
- **Ordre** (CarportOrders /admin-orders.html)
- **Materialer** (Materials /materials.html)

Flow for Carport Forespørgsler, Tilbud og Ordre

- Fra **Forespørgsler** kan sælgeren se en oversigt over forespørgsler og herfra tilgå en specifik anmodning (CarportRequestDetail /admin-request-detail.html).
- Herfra kan sælgeren redigere mål, stykliste og pris via (UpdateCarport /requests/{id}/update-carport.html). Det er her, sælgeren justerer pris og dækningsgrad før afsendelse af tilbud.
- Fra detaljesiden kan sælgeren sende tilbuddet eller åbne tegningen i et nyt vindue (CarportDrawing /show-carport-drawing/{id}).

- **Tilbud** giver adgang til en oversigt af aktive, afviste og udløbne tilbud samt detaljevisning (CarportOrderDetail /order-detail.html).
- **Ordre** giver adgang til en oversigt af betalte og annullerede ordre samt detaljevisning (CarportOrderDetail /order-detail.html).

Flow for Materialeadministration

- I **Materialer** kan sælgeren søge efter eksisterende materialer.
- Materialer kan opdateres via (UpdateMaterial /material-update.html) eller oprettes som nye via (CreateMaterial /material-create.html).
- Efter alle CRUD-handlinger (Create, Read, Update, Delete) returneres sælgeren til oversigten (Materials).

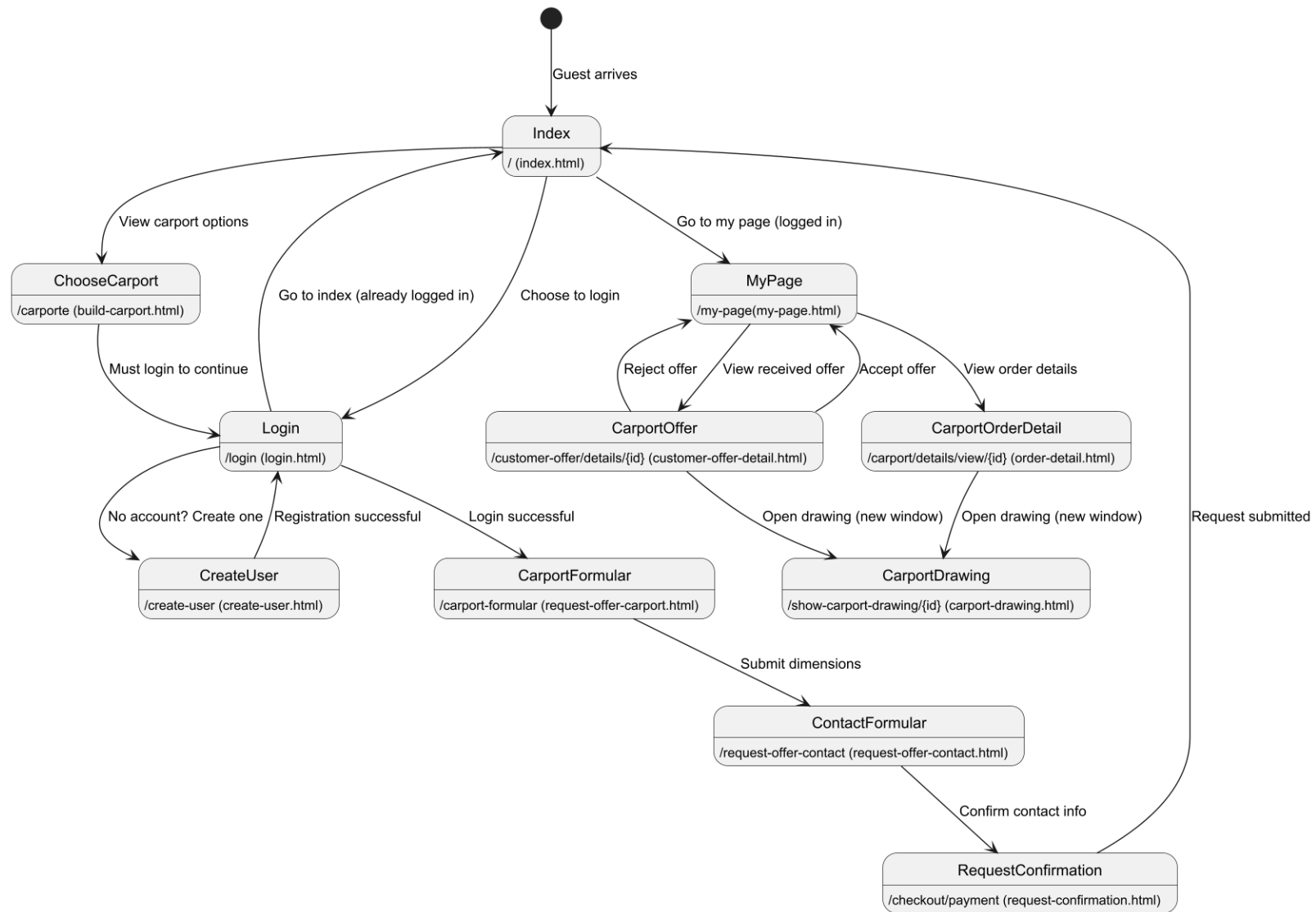
Fælles Navigationsbar (Header)

Alle sider i webapplikationen benytter sig af en fælles navigationsbar (header) for at sikre konsistent navigation. Indholdet af navigationsbaren justeres automatisk baseret på brugerens loginstatus og rolle:

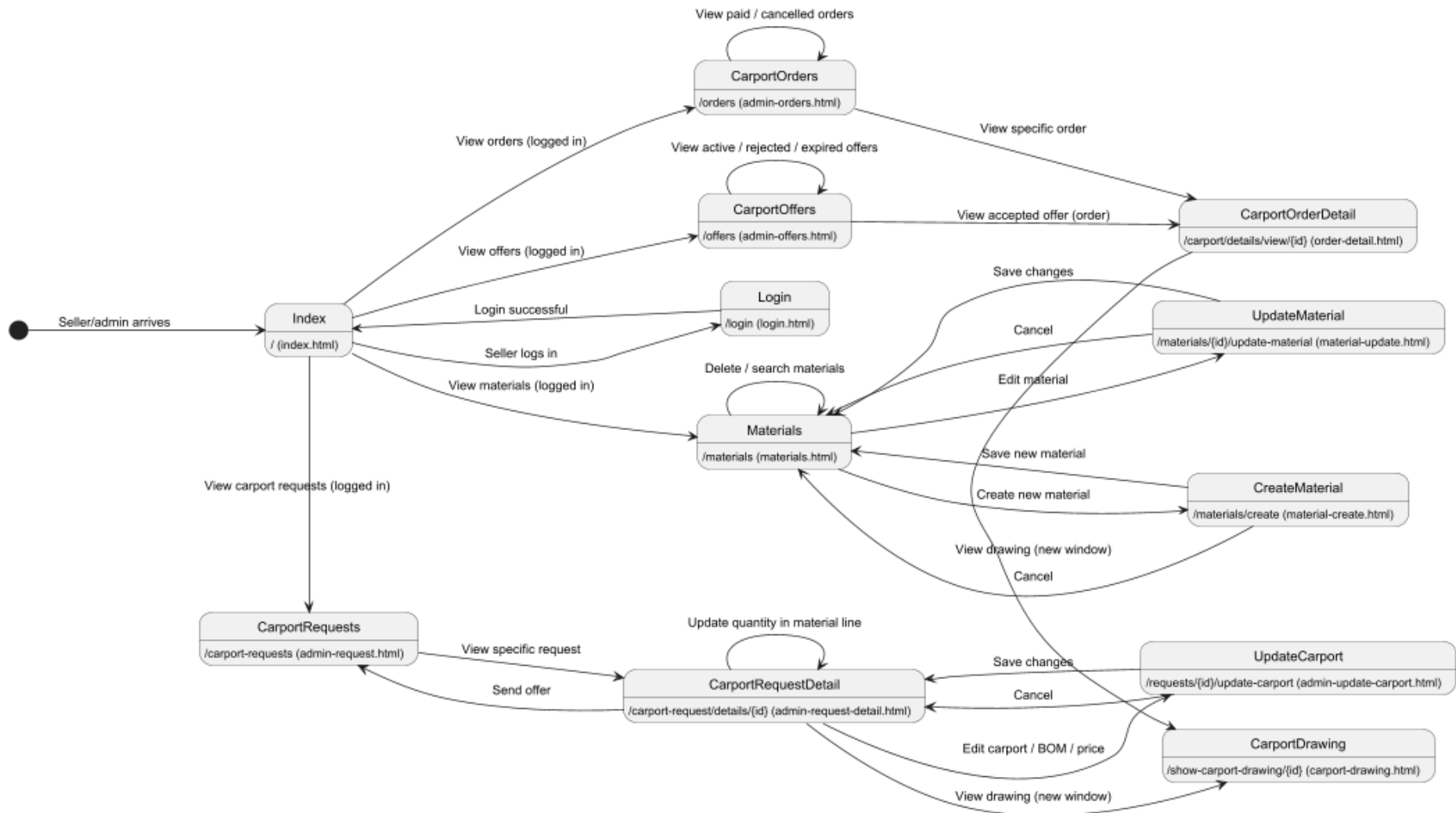
Rolle	Fælles	Rolle-Specifik
Gæst	Fog Logo (Index)	Login
Kunde (Logget ind)	Fog Logo (Index)	MyPage (Min side) Log Ud
Sælger (Logget ind)	Fog Logo (Index)	CarportRequests (Forespørgsler) CarportOffers (Tilbud) CarportOrders (Ordre) Materials (Materialer) Log ud

Begge diagrammer kan findes i GitHub repositoret under docs/models.

Navigationsdiagram – Kunde model



Navigationsdiagram – Sælger (ADMIN) model



Valg af arkitektur

Projektet er opbygget efter en MVC-inspireret arkitektur, udvidet med et servicelag. Arkitekturen er valgt for at opnå en klar adskillelse mellem præsentation, forretningslogik og dataadgang, samtidig med at applikationens kernefunktionalitet kan samles ét sted.

MVC med Servicelag

Controllerlaget håndterer HTTP-requests, routing og valg af view. Controllerne indeholder ikke forretningslogik, men fungerer som bindeled mellem weblaget og applikationens services. Viewlaget er implementeret med Thymeleaf og anvendes til præsentation af data i HTML, mens persistence-laget håndterer dataadgang og mapping til og fra databasen.

Det centrale arkitekturvalg i projektet er introduktionen af et servicelag, som ikke er en del af den klassiske MVC-model. Servicelaget er placeret mellem controller- og persistence-laget og indeholder applikationens forretningslogik, herunder beregninger, styklistegenerering, prisvalidering og opbygning af tegningsdata. Ved at samle denne logik i services holdes controllerne enkle, og forretningsregler kan ændres uden at påvirke præsentations- eller datalaget direkte. Servicelaget er desuden designet med fokus på testbarhed, da logikken kan testes uafhængigt af HTTP-laget.

Model-delen i arkitekturen er fordelt over domæneobjekter, servicelag og persistence-lag og er derfor ikke repræsenteret som ét selvstændigt lag i tabellen nedenfor.

Lag	Ansvar	Rolle
Controller	Håndterer HTTP-requests, routing og session.	Modtager request, kalder Service, vælger View
Service	Indeholder al forretningslogik og orkestrering.	Udfører beregninger og validering.
Mapper/ Persistence	CRUD-operationer og dataoverførsel til/fra databasen.	Omsætter domæneobjekter til databaseformat
View (Thymeleaf)	Præsentation af data.	Genererer HTML.

Applikationens struktur afspejles i pakkestrukturen, hvor hvert lag er adskilt i selvstændige pakker. Data overføres mellem lagene via Data Transfer Objects (DTO'er), og mapping anvendes for at undgå direkte afhængigheder mellem domænelaget og viewlaget. Hjælpeklasser (*util*) anvendes til stateless understøttende funktionalitet,

mens brugen af singletons kun anvendt i infrastrukturen, eksempelvis til ConnectionPool.

Pakke	Ansvar
config	Konfiguration af systemet (f.eks. session og Thymeleaf).
controllers	Modtager requests, kalder services, vælger view, håndterer validering på request-niveau.
dto	Data Transfer Objects, der sikrer løs kobling mellem lagene.
entities	Domæneobjekter (f.eks. Carport, Material, Order).
persistence	Dataadgang og mapping til/fra databasen.
services	Domæne- og forretningslogik (dimensionering, stykliste, pris/validering), orkestrerer datahåndtering.
util	"Stateless" hjælpefunktioner til beregning, formatering, validering (Helpers).

Arkitekturen med et separat servicelag medfører både fordele og ulemper, særligt i forhold til kompleksitet og test.

Fordele	Ulemper
Forretningslogik er samlet i servicelaget og adskilt fra controllerne.	Arkitekturen indeholder flere lag, hvilket øger kompleksiteten.
Controllerne kan holdes enkle og fokuseret på web-flow.	Mindre ændringer kan kræve justeringer i flere lag.
Forretningsregler kan ændres uden at påvirke views direkte.	Kræver mere struktur end en simpel MVC-løsning.
Logik kan testes uden afhængighed til web- og viewlaget.	Opsætning og forståelse af arkitekturen tager længere tid.

Særlige forhold

Session, request og application-variabler

Session

Systemet gemmer currentUser (UserDTO) i session efter login, hvilket reducerer gentagne databasekald og opretholder brugerens tilstand på tværs af requests. Sessionen anvendes desuden til flash messages (errorMessage, successMessage), som gemmes før en redirect og vises på destinationssiden, hvorefter de fjernes. Dette sikrer, at brugeren modtager feedback uden at beskeder hænger ved efterfølgende requests. Endelig anvendes loginRedirect til at gemme den ønskede destination, hvis en ikke-

logget bruger forsøger at tilgå en beskyttet side, eksempelvis oprettelse af en carportforespørgsel, som kræver login.

Request

Request-attributter anvendes til at sende data fra controller til view. Eksempelvis sendes `orderDetail`, `carportSvgTop` samt rollespecifikke attributter som `coveragePercentage` (kun for sælgere) til Thymeleaf-templates. Flash messages flyttes fra session til request i metoden `displayMessages()`, så de kan læses af viewet.

Exception-håndtering

Systemet anvender custom exceptions, herunder `DatabaseException` og `MaterialNotFoundException`, til at håndtere fejl på en kontrolleret og ensartet måde. Alle databaseoperationer i mapper-laget kan kaste `DatabaseException` ved SQL-fejl, som fanges i controller-laget og omsættes til brugervenlige fejlbeskeder, der gemmes i sessionen.

Eksempelvis vil et fejlslagent kald til `orderService.getOrderById()` resultere i, at brugeren redirectes med en besked som *“Kunne ikke hente ordre”*. Denne tilgang forhindrer, at tekniske fejl og stack traces vises for brugeren og sikrer en konsistent fejlhåndtering på tværs af systemet.

Der er ikke implementeret logging i projektets MVP. I en produktionsløsning vil det være hensigtsmæssigt at supplere med serverside logging, så fejl kan diagnosticeres uden at eksponere tekniske detaljer for brugeren.

Inputvalidering

Brugerinput valideres på to niveauer: i controller-laget og i service-laget.

I controller-laget valideres datatyper ved parsing, eksempelvis via `Integer.parseInt()`. Her fanges `NumberFormatException`, og brugeren præsenteres for en besked som *“Mål skal være et tal”*. Tomme eller null-værdier kontrolleres desuden eksplicit, inden input behandles videre.

I service-laget valideres forretningsregler. Eksempelvis sikrer `CarportService.validateCarport()`, at carportens dimensioner ligger inden for tilladte grænser (240–780 cm i længden og 240–600 cm i bredden), samt at et eventuelt skur ikke overstiger carportens størrelse. Hvis valideringen fejler, kastes en `IllegalArgumentException` med en beskrivende fejlmeddelelse, som vises for brugeren.

Passwords valideres ved registrering ved at kontrollere, at `password1` og `password2` matcher i `UserService.registerUser()`.

Sikkerhed ved login

Passwords gemmes aldrig i klartekst. Ved registrering hashes passwordet ved brug af BCrypt (BCrypt.hashpw(password, BCrypt.gensalt())), som anvender salt og flere hashing-runder for at beskytte mod eksempelvis rainbow table-angreb.

Ved login verificeres passwordet med BCrypt.checkpw(indtastetPassword, hashedPassword), som returnerer true, hvis passwordet matcher. Hvis valideringen fejler, kastes en DatabaseException med beskeden *“Forkert email eller password”*, hvilket forhindrer information leakage om, hvorvidt en given email eksisterer i systemet.

Sessioner sikres ved at invalidere hele sessionen ved logout (ctx.req().getSession().invalidate()). Derudover foretages der ejerskabstjek i controller-laget, eksempelvis ved at en kunde kun kan tilgå egne ordrer ved at sammenligne currentUser.userId() med order.getCustomerId().

Brugertyper (roller)

Systemet anvender to roller, defineret i Role-enum: CUSTOMER og SALESREP. Rollen gemmes i users-tabellen som en VARCHAR og mappes til Java via Role.valueOf(). Ved registrering tildeles nye brugere automatisk rollen CUSTOMER.

I controller-laget kontrolleres brugerens rolle, eksempelvis ved at tjekke currentUser.role().equals(Role.SALESREP), for at beskytte sælgerfunktionalitet. Forsøger en kunde at tilgå en sælger-side, redirectes brugeren til login med en fejlbesked.

Roller anvendes desuden til at styre, hvilke data der vises i views. Eksempelvis kan sælgere se kostpris og fortjeneste på ordrer, mens kunder udelukkende præsenteres for totalprisen.

Stykliste-beregner (BomService)

Styklisten beregnes i BomService, som implementerer interfacet IBomService. Når en carport oprettes, kaldes getBillOfMaterialByCarport(), som beregner nødvendige materialer ud fra carportens dimensioner.

For materialer, der er afhængige af længde (remme, spær, tagplader og brædder), vælges det optimale materiale via findOptimalVariantLength(). Metoden filtrerer materialer, der er lange nok, og vælger den kortest mulige længde for at minimere spild.

Eksempelvis vælges en 600 cm rem ved en carport på 570 cm, hvis de tilgængelige længder er 360, 480, 600 og 720 cm. Hvis ingen enkelt længde er tilstrækkelig, opdeles længden, og der anvendes to kortere materialer.

Antallet af materialekomponenter beregnes i utility-klassen `PartCalculator` baseret på konstruktionsregler, eksempelvis afstand mellem spær. Den samlede stykliste gemmes i databasen ved ordreoprettelse, så sælgeren kan redigere den før afsendelse af tilbud.

Tegning (SVG-generering)

Carport-tegninger genereres dynamisk som SVG (Scalable Vector Graphics) via klasserne `CarportSvgTop` og `CarportSvgSide`, som anvender fælles hjælpefunktioner fra `Svg`. Tegningerne opbygges som SVG-markup bestående af elementer som `<rect>`, `<line>` og `<polygon>`, der repræsenterer carportens komponenter.

I designfasen blev det overvejet, om tegningerne skulle gemmes i databasen eller genereres dynamisk. Valget faldt på runtime-generering, da dette giver en fleksibel løsning, hvor ændringer i carportens mål straks afspejles i tegningen, uden at data skal lagres eller vedligeholdes separat. Samtidig er tegneren løst koblet og udelukkende afhængig af carportens dimensioner, hvilket gør løsningen modulær og nem at ændre. Ulempen er et øget beregningsarbejde ved visning, men dette blev vurderet acceptabelt i forhold til fordelene.

Ved visning af en ordre kaldes `carportService.getCarportTopSvgView(carport)`, som beregner koordinater for stolper, remme, spær og hulbånd ud fra carportens dimensioner. Hvis carporten indeholder et skur, tegnes dette baseret på `ShedPlacement` og skurets mål.

Den genererede SVG sendes direkte til Thymeleaf-viewet via `th:utext`, hvor den renderes i browseren uden brug af eksterne filer. Dette giver en skalerbar og præcis tegning, som kan zoomes uden kvalitetstab og altid matcher den konkrete carport.

AppProperties – centraliseret konfiguration

Systemet anvender en `AppProperties`-klasse til central håndtering af konfigurationsværdier, som indlæses fra filen `app.properties`. Formålet er at undgå hardcodede værdier i koden og gøre systemet let at konfigurere uden genkompilering.

Konfigurationsfilen indeholder både forretningsregler (f.eks. dækningsgrad), konstruktionsregler for carporte (maksimal stolpeafstand, standardlængder) samt parametre til SVG-tegning. Filen indlæses ved applikationsstart via en statisk initialisering, og manglende eller ugyldige værdier medfører en `PropertyException`.

Klassen stiller metoder til rådighed for hentning af påkrævede værdier som `String`, `int` og `double`, hvor alle validerer korrekt eksistens og parsing. Af sikkerhedshensyn er `app.properties` udeladt fra versionsstyring og erstattet af en `app.properties.example`, som dokumenterer strukturen uden at indeholde forretningskritiske værdier.

Denne tilgang gør det nemt at justere centrale regler, eksempelvis ændring af dækningsgrad, ved kun at redigere én konfigurationsfil.

Email-integration (SendGrid)

Systemet anvender SendGrid til automatisk udsendelse af emails til kunder i forbindelse med forespørgsel, tilbud og ordrebekræftelse. Der benyttes tre faste email-skabeloner, som dækker disse hændelser og sikrer ensartet kommunikation med kunden.

Emails sendes via et dedikeret servicelag, hvor dynamiske data som kundenavn, ordreoplysninger og pris indsættes i foruddefinerede skabeloner. Selve HTML-layout og indhold er adskilt fra koden og fremgår af Bilag 5: Kunde e-mails.

API-nøgle til SendGrid håndteres via miljøvariabler, så følsomme oplysninger ikke indgår i versionsstyring.

Sammenfatning

Systemet benytter sessioner til autentificering og brugerfeedback, kontrolleret fejlhåndtering via custom exceptions, flerlaget inputvalidering samt sikker håndtering af passwords og roller. Derudover anvendes algoritmisk styklisteberegning, dynamisk SVG-generering, centraliseret konfiguration og automatisk email-afsendelse.

Kodeeksempler

I dette kapitel vises udvalgte kodeeksempler, der implementerer central forretningslogik fra de udvalgte user stories. Eksemplerne er grupperet i tre flows og illustrerer sammenhængen mellem krav, user stories og konkrete tekniske løsninger.

Kodeeksempler er layout-tilpasset rapportformatet, men svarer funktionelt til den implementerede kode.

Kunde-flow – Fra tilbud til ordre

Dette afsnit beskriver kunde-flowet fra det tidspunkt, hvor et tilbud er klar, til kunden har accepteret tilbuddet og kan tilgå sin betalte ordre. Flowet tager udgangspunkt i de krav, der regulerer hvilket indhold kunden må se før og efter betaling, og er realiseret gennem user stories US-6, US-7 og US-10, som tilsammen udgør ét sammenhængende forløb.

En central del af forretningslogikken er, at styklisten først må være tilgængelig efter betaling. Dette er løst ved at modellere ordreflowet eksplicit via en OrderStatus-enum, som anvendes konsekvent på tværs af controller-, service- og view-lag til at styre, hvilke data der må vises for kunden. Ordren bevæger sig gennem veldefinerede stadier, herunder PENDING, READY og PAID, hvilket sikrer et entydigt og kontrolleret flow.

Når en kunde opretter en forespørgsel, oprettes der automatisk en ordre med status PENDING. Denne status anvendes udelukkende til sælgerens overblik over nye forespørgsler. Først når sælgeren har gennemgået forespørgslen og aktivt godkendt tilbuddet, opdateres ordren til READY, hvorefter kunden kan se tilbuddet og tage stilling til det.

```

public Order createOrder(Connection connection, 3 usages  ⚡ Morten Jensen +1 *
    int customerId,
    int carportId,
    String customerComment,
    PricingDetails pricingDetails
) throws DatabaseException
{
    String sql = """
        INSERT INTO "orders" (customer_id,
                               carport_id,
                               customer_comment,
                               order_status,
                               coverage_percentage,
                               cost_price
                               )
        VALUES (?, ?, ?, 'PENDING', ?, ?)
        RETURNING order_id, request_created_at, order_status
        """;

```

Eksempel 1: Ved oprettelse af ordre i forbindelse med forespørgsel, sættes status til PENDING

For at adskille kundens og sælgerens adgang til data anvendes separate objekter. Når en kunde modtager et tilbud, består dette af en CustomerOfferDTO, som indeholder oplysninger om carport, tegning og pris, men aldrig styklisten. Sælgeren arbejder derimod med et mere fyldigt OrderDetail-objekt, som indeholder alle relevante oplysninger om ordren.

```

public record CustomerOfferDTO( 9 usages  ⚡ DHangaard
    int orderId, no usages
    Carport carport, 2 usages
    UserDTO customer, no usages
    UserDTO seller, no usages
    OrderTimeLine orderTimeLine, no usages
    Double totalPrice, 1 usage
    OrderStatus orderStatus, no usages
    String customerComment no usages
)
{
}

```

Eksempel 2: CustomerOfferDTO indeholder ikke en stykliste (BillOfMaterials)


```
private void showCustomerOfferDetails(Context ctx) 1 usage 2 DHangaard +1 *
{
    if (!requireLogin(ctx))
    {
        return;
    }

    displayMessages(ctx);

    int orderId = Integer.parseInt(ctx.pathParam(s: "id"));
    Locale.setDefault(new Locale(language: "US"));

    try
    {
        CustomerOfferDTO offer = orderService
            .getCustomerOfferByOrderId(orderId);
        CarportSvgTop carportSvgTop = carportService
            .getCarportTopSvgView(offer.carport());
        CarportSvgSide carportSvgSide = carportService
            .getCarportSideSvgView(offer.carport());

        ctx.attribute("carportSvgTop", carportSvgTop);
        ctx.attribute("carportSvgSide", carportSvgSide);
        ctx.attribute("offer", offer);
        ctx.attribute("formattedTotalPrice", PriceFormatUtil
            .getFormattedPrice(offer.totalPrice()));

        ctx.render(filePath: "customer-offer-detail.html");
    }
}
```

Eksempel 3: Ved brug af CustomerOfferDTO sikres det, at kunden aldrig modtager styklisten før betaling, selv hvis der sker fejl i view-laget.

Når kunden accepterer tilbuddet og gennemfører betalingen, opdateres ordren til status PAID. Først på dette tidspunkt frigives styklisten, og kunden får adgang til den fulde ordre.

```
try
{
    UserDTO customer = ctx.sessionAttribute( key: "currentUser");
    int userId = customer.userId();

    List<OrderOverviewDTO> pendingOrders = orderService
        .getAllOrdersByUserIdAndStatus(userId, OrderStatus.PENDING);
    List<OrderOverviewDTO> readyOrders = orderService
        .getAllOrdersByUserIdAndStatus(userId, OrderStatus.READY);
    List<OrderOverviewDTO> paidOrders = orderService
        .getAllOrdersByUserIdAndStatus(userId, OrderStatus.PAID);

    ctx.attribute("pendingOrders", pendingOrders);
    ctx.attribute("readyOrders", readyOrders);
    ctx.attribute("paidOrders", paidOrders);
}
```

Eksempel 4: Ordren hentes entydigt fra databasen baseret på user-id og status, hvilket sikrer korrekt adgangsstyring – her ses forespørgsel (PENDING), tilbud (READY) og ordre (PAID).

Opdatering af materialer i en stykliste

Dette afsnit beskriver, hvordan sælgeren kan justere materialer i en eksisterende stykliste som led i den rådgivende salgsproces. Flowet udspringer af Krav 2 og Krav 8, hvor automatiseret beregning kombineres med mulighed for manuel tilpasning foretaget af sælgeren.

Når en sælger justerer en ordre – eksempelvis ved at ændre carportens dimensioner, tilføje eller fjerne et skur eller korrigere mængden af et materiale – genberegnes både stykliste, pris og tegning. Ændringerne gemmes straks i databasen, så ordren altid afspejler den aktuelle tilstand.

I SellerController håndteres opdateringen af en materialelinje. Når sælgeren foretager en ændring, kaldes MaterialService for at genberegne den samlede pris for ordren og derefter opdatere materialelinjen.

```
double lineTotalDifference = materialService
    .calculateLinePriceDifference(materialLineId, quantity);
boolean updated = materialService
    .updateBillOfMaterialLineQuantity(materialLineId, quantity);
```

Eksempel 5: Genberegning af pris og opdatering af materialelinje.

```

@Override 1 usage Morten Jensen *
public double calculateLinePriceDifference(int materialLineId,
                                           int quantity
) throws DatabaseException
{
    ValidationUtil.validateQuantity(quantity);

    MaterialLine materialLine = materialLineMapper
        .getMaterialLineById(materialLineId);
    double oldMaterialLineTotal = calculateLineTotal(materialLine);
    double newMaterialLineTotal = materialLine.getMaterialVariant()
        .getUnitPrice() * quantity;

    return newMaterialLineTotal - oldMaterialLineTotal;
}

```

Eksempel 6: En materialelinjes totalpris genberegnes

Hvis opdateringen gennemføres korrekt, kaldes OrderService, som opdaterer ordren via OrderMapper. Den nye kostpris gemmes i databasen, så både kunde- og sælgerdata forbliver konsistente.

```

if (updated)
{
    Order order = orderService.getOrderById(orderId);
    double orderCostTotal = order.getPricingDetails().getCostPrice();
    double newOrderCostTotal = orderCostTotal + lineTotalDifference;

    orderService.updateOrderCostPrice(orderId, newOrderCostTotal);
    ctx.sessionAttribute("successMessage", "Antal opdateret");
}

```

Eksempel 7: Ordren opdateres i databasen med ny kostpris

Denne tilgang sikrer, at sælgeren kan foretage nødvendige justeringer i dialog med kunden, uden at kompromittere automatiseringen eller den samlede prisberegning.

Opdatering af materialer i databasen

Dette afsnit beskriver, hvordan sælgere kan administrere materialer i databasen som led i systemets understøttelse af fulde CRUD-operationer. Funktionaliteten udspringer af Krav 3 og sikrer, at priser, dimensioner og beskrivelser altid er opdaterede og konsistente.

Adgangen til materialehåndtering er begrænset til sælgere. Via en dedikeret side kan sælgeren søge efter materialer baseret på ID, navn, kategori eller type. Søgeresultatet viser eksisterende materialer, som kan opdateres eller slettes, ligesom nye materialer kan oprettes fra bunden via en separat formular.

MaterialController håndterer input fra formularerne og opbygger et MaterialVariant-objekt baseret på brugerens indtastninger.

```
try
{
    MaterialVariant oldMaterialVariant = materialService
        .getMaterialVariantById(materialVariantId);

    MaterialVariant updatedMaterialVariant = buildVariantFromForm(ctx);
    Material updatedMaterial = updatedMaterialVariant
        .getMaterial();
    updatedMaterial.setMaterialId(oldMaterialVariant
        .getMaterialId());

    updatedMaterialVariant.setMaterialVariantId(oldMaterialVariant
        .getMaterialVariantId());
    updatedMaterialVariant.setMaterialId(oldMaterialVariant
        .getMaterialId());
    updatedMaterialVariant.setMaterial(updatedMaterial);

    boolean isUpdated = materialService
        .updateMaterialVariant(updatedMaterialVariant);

    if(isUpdated)
    {
        ctx.sessionAttribute("successMessage",
            "Du har opdateret material varianten med id: " +
            updatedMaterialVariant
                .getMaterialVariantId());
    }
    else
    {
        ctx.sessionAttribute("errorMessage",
            "Noget gik galt ved opdatering af materiale varianten med id: " +
            updatedMaterialVariant
                .getMaterialVariantId());
    }
    ctx.redirect(location: "/materials");
}
```

Eksempel 8: Brugerinput bliver til en materialevariant

I MaterialService valideres data, hvorefter både Material og MaterialVariant opdateres i databasen via MaterialMapper og MaterialVariantMapper.

```

try
{
    isUpdated = materialMapper.updateMaterial(
        connection,
        variant.getMaterial()
    );

    isUpdated = materialVariantMapper.updateMaterialVariant(
        connection,
        variant
    );
}

```

Eksempel 9: Materiale samt materialevariant opdateres i databasen

Hvis opdateringen gennemføres korrekt, præsenteres sælgeren for en succesbesked i webapplikationen. Ved fejl vises en tilsvarende fejlbesked, hvilket sikrer tydelig feedback til brugeren.

Denne løsning sikrer, at materialedata altid afspejler den aktuelle forretningsvirkelighed, samtidig med at adgangen er rollebaseret og begrænset til sælgere for at opretholde dataintegritet og sikkerhed.

Beregning af stykliste – teknisk flow

Dette afsnit beskriver den tekniske implementering af styklisheberegningen, som udgør en central del af systemets forretningslogik. Flowet dækker både den automatiske generering af materialer og den efterfølgende lagring af stykliste og pris i databasen.

Når kunden har udfyldt carportformularen, beregnes styklisten automatisk, men kan stadig justeres af en sælger efterfølgende.

Styklisheberegningen håndteres i BomService, som implementerer interfacet *IBomService*. Servicen fungerer som et koordinerende lag, der samler resultater fra flere hjælpekomponenter, herunder PartCalculator og MaterialVariantMapper.

```

public interface IBomService 5 usages 1 implementation 2 DHangaard +1
{
    public List<MaterialLine> getBillOfMaterialByCarport( 32 usages 1 implementation 2 DHangaard
        Carport carport
    ) throws DatabaseException, MaterialNotFoundException;

    public PricingDetails calculateCarportPrice( 3 usages 1 implementation 2 DHangaard
        List<MaterialLine> billOfMaterial
    );
}

```

Eksempel 10: Kald til BomService, hvor styklisheberegningen initieres

De konkrete beregninger udføres i PartCalculator, som indeholder al logik relateret til dimensioner og konstruktionsregler. Klassen anvender blandt andet standardlængder, afstandskrav, buffer-tolerancer og skurplacering til at beregne antal stolper, spær, tagplader, beslag og skruepakker

```
private MaterialLine calculateNumberOfPosts(Carport carport) throws DatabaseException, MaterialNotFoundException
{
    int numberOfPosts = 0;

    if (carport.getShed() != null)
    {
        numberOfPosts = PartCalculator.calculateNumberOfPostsWithShed(carport.getLength(), carport.getShed());
    }
    else
    {
        numberOfPosts = PartCalculator.calculateNumberOfPostsWithoutShed(carport.getLength());
    }

    List<MaterialVariant> posts = variantMapper.getAllVariantsByType(MaterialType.POST);

    MaterialVariant postVariant = posts.stream() Stream<MaterialVariant>
        .filter( MaterialVariant materialVariant -> materialVariant.getVariantLength() == STANDARD_POST_LENGTH)
        .findFirst() Optional<MaterialVariant>
        .orElseThrow(() -> new MaterialNotFoundException("Kunne ikke finde stolpe"));

    return new MaterialLine(postVariant, numberOfPosts);
}
```

Eksempel 11: Beregning af antal stolper baseret på en carports mål

Når der skal vælges den mest hensigtsmæssige længde på et materiale eller dets maksimale længde, anvendes hjælpefunktioner, der arbejder direkte mod varekatalogets tilgængelige varianter i databasen.

```
private MaterialVariant findOptimalVariantLength(List<MaterialVariant> variants, 10 usages 2 DHangaard *
                                                int length
) throws MaterialNotFoundException
{
    return variants.stream() Stream<MaterialVariant>
        .filter( MaterialVariant variant -> variant.getVariantLength() != null)
        .filter( MaterialVariant variant -> variant.getVariantLength() >= length)
        .min(Comparator.comparingInt(MaterialVariant::getVariantLength)) Optional<MaterialVariant>
        .orElseThrow(() -> new MaterialNotFoundException("Ingen kombination af materialer passer til længde: "
            + length
            + " cm."));
}

private int getMaxVariantLength(List<MaterialVariant> variants) throws MaterialNotFoundException 6 usages 2 DHangaard
{
    return variants.stream() Stream<MaterialVariant>
        .filter( MaterialVariant materialVariant -> materialVariant.getVariantLength() != null)
        .mapToInt(MaterialVariant::getVariantLength) IntStream
        .max() OptionalInt
        .orElseThrow(() -> new MaterialNotFoundException("Ingen materialer fundet"));
}
```

Eksempel 12: Hjælpefunktioner til hhv. valg af materialelængde og bestemmelse af en materialevariants maksimale længde

Metoden `findOptimalVariantLength()` anvendes i flere beregningsmetoder, herunder *calculateBoards*, hvor den bruges til at bestemme antal stern- og vandbrædder. Metoden sikrer, at materialerne dækker carportens dimensioner korrekt, samtidig med at spild minimeres. Den maksimale tilgængelige længde, fundet via `findMaxVariantLength()`, anvendes til at styre flowet i beregningen, herunder hvornår en længde må opdeles i flere dele.

```
private MaterialLine calculateBoards(List<MaterialVariant> variants, 6 usages 2 Morten Jensen *
    int carportDimensionInCm,
    int maxVariantLength,
    int numberOfSidesOrEnds
) throws MaterialNotFoundException
{
    int dimensionToCover = carportDimensionInCm + BOARD_BUFFER_TOLERANCE_CM;

    if (dimensionToCover <= maxVariantLength)
    {
        MaterialVariant singleBoardVariant = findOptimalVariantLength(variants, dimensionToCover);
        return new MaterialLine(singleBoardVariant, numberOfSidesOrEnds);
    }
    else
    {
        int halfDimension = (carportDimensionInCm / 2) + BOARD_BUFFER_TOLERANCE_CM;
        MaterialVariant splitBoardVariant = findOptimalVariantLength(variants, halfDimension);

        return new MaterialLine(splitBoardVariant, quantity: numberOfSidesOrEnds * 2);
    }
}
```

Eksempel 13: *calculateBoards* med hjælpefunktioner

Når styklisten er beregnet, gemmes både materialer og pris i databasen via `OrderService`.

```
List<MaterialLine> bom = bomService.getBillOfMaterialByCarport(savedCarport);
PricingDetails pricingDetails = bomService.calculateCarportPrice(bom);
```

Eksempel 14: Materialer og pris gemmes i databasen

Metoden *createPendingOrder* opretter en ny ordre, når kunden laver en forespørgsel. Her beregnes styklisten via `BomService`, hvorefter ordren gemmes med kundeinformation, carport-id, kommentar og samlet pris. Hver materialelinje gemmes som en *MaterialLine* i databasen. Hele processen afvikles i én database-transaktion for at sikre konsistens mellem ordre, stykliste og pris.

```
Order savedOrder = orderMapper.createOrder(  
    connection,  
    customerId,  
    savedCarport.getCarportId(),  
    customerComment,  
    pricingDetails  
);  
  
for(MaterialLine line: bom)  
{  
    materialLineMapper.createMaterialLine(  
        connection,  
        savedOrder.getOrderId(),  
        line.getMaterialVariant().getMaterialVariantId(),  
        line.getQuantity()  
    );  
}
```

Eksempel 15: Styklisten oprettes i databasen

De udvalgte kodeeksempler illustrerer, hvordan overordnede forretningskrav er omsat til konkrete user stories og videre til tekniske løsninger. Ved at adskille ansvar mellem beregning, materialevalg og persistens opnås et robust og vedligeholdbart flow, hvor stykliste og pris altid afspejler carportens faktiske konfiguration.

Status på implementering

Projektet er implementeret som en MVP (Minimum Viable Product) med fokus på kernefunktionalitet og central forretningslogik frem for fuld funktionel dækning og visuel færdiggørelse. De primære flows for kundeforespørgsel, automatisk stykliste- og prisberegning samt sælgerens håndtering af tilbud er implementeret og funktionsdygtige.

Der er dog en række områder, som enten er fravalgt eller ikke fuldt implementeret inden aflevering:

Login-krav ved forespørgsel

En kunde kan ikke sende en forespørgsel på en carport uden at være logget ind. Dette er et bevidst valg i prototypen for at kunne demonstrere flest mulige funktioner relateret til brugerroller, sessions og ordreflow. Funktionalitet for forespørgsler uden login er ikke nået inden for projektets tidsramme.

Ordrebekræftelse via e-mail

Ordrebekræftelser sendes som e-mails, men indeholder udelukkende data og ikke vedhæftede filer såsom tegning eller stykliste. En mere fuldt udbygget løsning ville inkludere generering og vedhæftning af relevante dokumenter som PDF.

Begrænset redigering af stykliste

En sælger eller administrator kan justere mængder og priser i styklisten, men kan ikke frit udskifte materialer med alternative varianter ud over dem, som BomService automatisk vælger. Det er eksempelvis ikke muligt at vælge en helt anden stolpetype end den beregneren foreslår.

Manglende understøttelse af carport med rejsning

Systemet understøtter udelukkende carporte med fladt tag. Beregning og visualisering af carporte med rejsning er ikke implementeret, da dette lå uden for projektets afgrænsning.

Forenklet databasestruktur

Databasen håndterer ikke lagerstyring, såsom beholdning, reservation eller automatisk nedskrivning af materialer ved køb. En mere realistisk løsning ville inkludere lagerstatus og genbestillingslogik.

Styling og responsivt design

Mobilvenlighed for sælger-views og dashboards er ikke prioriteret. Applikationen er primært designet til desktopbrug, da det antages, at sælgere håndterer komplekse tilbud fra en arbejdsstation. Mobilunderstøttelse for sælgerrollen vurderes som et oplagt næste udviklingstrin.

Kvalitetssikring (test)

Automatiserede tests

Test har været en central del af udviklingsprocessen. Hvor det har været muligt, er der arbejdet ud fra en Test Driven Development (TDD)-tilgang, særligt ved implementering af kritiske dele af systemet såsom beregning af styklister, priser og validering af input. Formålet har været at sikre korrekt forretningslogik samt høj stabilitet i systemet.

Der er anvendt både unit tests og integrationstests.

Unit tests er anvendt til at teste afgrænsede og uafhængige util-klasser, hvor logikken kan verificeres isoleret. Integrationstests er anvendt på service- og persistence-lagene for at sikre, at data korrekt kan oprettes, hentes, opdateres og slettes i databasen. Dette har været essentielt, da systemet er stærkt afhængigt af korrekt håndtering af materialer, styklister og ordreda.

Test af styklisterberegning (kritisk forretningslogik)

En væsentlig del af testarbejdet har været fokuseret på beregning af styklister, da denne funktion er forretningskritisk for både kunder og sælgere. Her er der blandt andet testet:

- at den korrekte materialetype vælges
- at den mest hensigtsmæssige længde findes blandt de tilgængelige materialer i databasen
- at materialer opdeles korrekt, hvis én enkelt længde ikke er tilstrækkelig
- at der tages højde for buffer og tolerance for at minimere spild og sikre korrekt montage

Dette sikrer både korrekt prisberegning og understøtter Fogs bæredygtighedsprofil ved at minimere materialespild.

BomService er testet grundigt via integrationstests, da klassen både indeholder kompleks forretningslogik og afhænger af data fra databasen. Testene dækker blandt andet prisberegning baseret på materialelinjer, valg af korrekt længde på spær, remme og tagplader ud fra carportens dimensioner, håndtering af carporte med og uden skur samt korrekt beregning af sternbrædder, vandbrædder og under- og overstern.

Teststrategi

Der er arbejdet systematisk med test før kode blev merged til developer- og main-branch. Nye features er udviklet på separate branches, og tests er skrevet før eller parallelt med implementeringen. Kode er først blevet merged, når alle relevante tests har passeret. Denne tilgang har reduceret fejl, øget kodekvaliteten og gjort systemet mere robust over for ændringer.

Nedenstående tabel giver et overblik over de automatiserede unit- og integrationstests, herunder hvilke klasser der er testet, testdækning af klassernes metoder samt formålet med testene.

Test type	Klasse	Operation	Dækning	Formål
Unit	PartCalculator	Beregning af antal pr. materialetype	12/12	Sikrer korrekt beregning af materialemængder, uafhængigt af øvrig logik
Unit	PostPlacement Calculator	Beregning af stolpeplacering	3/11	Validerer korrekt placering af stolper til tegning og konstruktion
Unit	ValidationUtil	Input-validering	8/11	Sikrer at brugerinput overholder forretningsregler
Integration	BomService	Stykliste-beregning Prisberegning	21/21	Tester samlet stykliste- og prislogik op mod databasen
Integration	Persistence-lag (Mappers)	Database-operationer	65/71	Validerer korrekt interaktion med databasen

Automatiserede tests har samlet set spillet en afgørende rolle i udviklingen af systemet. Ved konsekvent at anvende unit- og integrationstests er det sikret, at både forretningslogik og datatilgang fungerer korrekt og stabilt. Særligt beregningen af styklister og priser har været kritisk, da fejl her direkte vil påvirke både kunder og sælgere. Derfor er disse dele af systemet testet grundigt gennem integrationstests, som verificerer korrekt samspil mellem service-lag, persistence-lag og databasen.

Ved at anvende TDD på centrale komponenter er der opnået klart afgrænsede og testbare klasser, og brugen af uafhængige util-klasser har gjort det muligt at teste komplekse beregninger isoleret. Den systematiske testproces har samtidig fungeret som et kvalitetssikringsværktøj før merge til developer- og main-branch, hvor kun kode med beståede tests er blevet integreret.

User Acceptance tests

User story	Status	Bemærkning
1	Godkendt	
2	Godkendt	
3.1	Godkendt	

User story	Status	Bemærkning
3.2	Ikke godkendt	En kunde kan ikke vælge tagtypen "Med rejsning", eller forskellige materialer
3.3	Godkendt	
4	Godkendt	
5	Godkendt	
6	Godkendt	
7	Godkendt	
8	Godkendt	
9	Godkendt	
10	Godkendt	
11	Godkendt	
12	Godkendt	
13	Godkendt	
14	Godkendt	
15	Godkendt	
16	Delvist godkendt	En sælger kan ikke ændre et specifikt materiale, kun rette i mængden på dette, samt tilføje nye materialer til styklisten
17	Godkendt	
18	Godkendt	
19	Godkendt	
20	Godkendt	
21	Godkendt	
22	Godkendt	
23	Godkendt	
24	Delvist godkendt	Materialer kan slettes, men dette har konsekvenser for tidligere og igangværende ordre der indeholder dette specifikke materiale. Derfor er der i databasen sat en constraint og denne metode kaster en Database Exception, men denne er ikke håndteret yderligere. Materialevarianten kan slettes hvis den ikke er afhængig af en stykliste.
25	Godkendt	

Proces

Dette kapitel redegør for projektets arbejdsproces. Først beskrives, hvordan arbejdet konkret er blevet planlagt og gennemført, hvorefter der reflekteres over samarbejde, udfordringer og læringspunkter fra forløbet.

Arbejdsprocessen faktisk

Projektet i faser

Projektforløbet kan overordnet opdeles i tre naturlige faser, som tilsammen strakte sig over fem sprints. Den detaljerede planlægning af sprints, opgaver og user stories fremgår af Bilag 6: Detaljeret plan, mens faserne her beskrives på et mere overordnet niveau for at give et samlet overblik over arbejdsprocessen.

Den første fase bestod af ét sprint og havde fokus på opstart af projektet. I denne periode blev der arbejdet med analyse, design og etablering af projektets tekniske og organisatoriske rammer. Herunder blev kundemateriale gennemgået, centrale modeller udarbejdet, og projektets grundlæggende struktur sat op.

Den anden fase udgjorde projektets hoveddel og strakte sig over tre sprints. Denne fase omfattede den egentlige udvikling af systemet, hvor der blev arbejdet med implementering af funktionalitet på tværs af samtlige user stories. Fokus var her på kodning, løbende justering af løsningen samt integration mellem de forskellige dele af systemet i overensstemmelse med den fastlagte plan.

Den tredje og afsluttende fase bestod af ét sprint og havde fokus på færdiggørelse af projektet. I denne periode blev der arbejdet med rapportskrivning, afsluttende justeringer af systemet samt deployment. Formålet var at samle projektet til et færdigt produkt og sikre, at både dokumentation og løsning var klar til aflevering.

Sprints havde gennem hele projektet en varighed på cirka én uge og var struktureret omkring vejledningsmøderne, som typisk fandt sted om mandagen. Disse møder fungerede som naturlige skillelinjer mellem sprints og var med til at styre prioritering og fremdrift i de enkelte faser.

Kanban

Rollen som Scrum Master (benævnt således i projektet, men svarende til Kanban-ansvarlig) har været fordelt på skift mellem gruppens medlemmer gennem projektperioden. Fordelingen fremgår af bilag 6, hvor det fremgår, hvem der varetog rollen i de enkelte sprints. Formålet med denne rotation var at give alle gruppemedlemmer erfaring med procesansvar og overblik over projektets fremdrift.

Inden kodestart blev Scrum Masterens ansvarsområde fastlagt i et fælles codingstandards.md-dokument. Rollen havde ansvar for at sikre overholdelse af den

aftalte arbejdsproces samt for at prioritere opgaver ved fravær i teamet. Derudover havde Scrum Masteren ansvar for en række kritiske procedurer, som er identificeret i risikoanalysen, herunder håndtering af opgavestatus og koordinering ved forsinkelser eller uforudsete hændelser.

I praksis var Scrum Masteren primært ansvarlig for vedligeholdelse af GitHub Projects. Dette omfattede opdatering af Kanban-boardet i forbindelse med ugens opstart, typisk efter mandagsmødet, samt en afsluttende opdatering mod slutningen af ugen. Denne struktur bidrog til, at gruppen havde et fælles og opdateret overblik over igangværende og afsluttede opgaver.

Selvom rollen ikke har været knyttet til én enkelt hændelse, der i sig selv har været afgørende for projektets forløb, har den løbende varetagelse af Scrum Master-rollen bidraget til stabilitet i processen og gjort det lettere at reagere på mindre udfordringer undervejs.

Vejledning

I projektets anden uge blev der afholdt et vejledningsmøde med fokus på databasedesign og ER-diagram. Forud for mødet havde gruppen identificeret en mulig udfordring i form af et relativt komplekst databaseskema, herunder et højt antal 1:1-relationer, som kunne få betydning for den videre udvikling.

Denne vurdering blev bekræftet under vejledningen, og på den baggrund blev det besluttet at refaktorere databasedesignet. Refaktoreringen omfattede både en forenkling af tabeller og relationer samt tilpasning af de tilhørende mappers. Resultatet var et mere overskueligt og modulært design, som gav et bedre grundlag for den efterfølgende implementering.

Kommunikation

Kommunikationen i teamet har primært foregået ved fysisk tilstedeværelse på skolen, hvor gruppen har arbejdet sammen på daglig basis. Dette har muliggjort løbende dialog samt korte morgenmøder og daglige afrundinger, som har skabt en klar fælles forståelse af opgaver og fremdrift.

I de tilfælde, hvor arbejdet foregik remote, eksempelvis ved sygdom, blev Discord anvendt som primær kommunikationsplatform. Herudover blev GitHub anvendt aktivt, særligt GitHub Projects, til at skabe overblik over igangværende opgaver og pull requests. Samlet set har denne kombination af fysisk samarbejde og digitale værktøjer bidraget til en ensartet og effektiv kommunikation gennem hele projektførelsen.

Arbejdsprocessen reflekteret

Scrum master

Rollen som Kanban-ansvarlig (Scrum master) fungerede overordnet efter hensigten og bidrog til struktur og overblik i projektet. Anvendelsen af GitHub Projects efter Kanban-principper gjorde det muligt løbende at følge opgavernes status og skabe en fælles forståelse af fremdrift og prioriteringer.

Samtidig oplevede gruppen udfordringer i forbindelse med opbygningen af backloggen, særligt i projektets tidlige fase. Når et projekt starter fra bunden uden eksisterende kode eller reference, kan det være vanskeligt at nedbryde funktionalitet i meningsfulde tasks og vurdere deres omfang. I takt med at projektet udviklede sig, blev flere opgaver derfor justeret eller omskrevet, efterhånden som krav og design blev tydeligere.

For at imødekomme disse udfordringer blev backloggen behandlet som et dynamisk arbejdsredskab frem for en fast plan. Opgaver blev løbende justeret, opdelt eller omformuleret i takt med, at forståelsen af systemets krav og struktur blev mere tydelig. Denne tilgang gjorde det muligt at bevare overblik og fremdrift, samtidig med at planlægningen tilpassede sig projektets naturlige udvikling.

Disse erfaringer peger ikke på begrænsninger i Kanban som metode, men snarere på betydningen af erfaring og domæneforståelse ved planlægning af et projekt. Trods udfordringerne har Kanban-arbejdsformen bidraget positivt ved at skabe gennemsigtighed, forudsigelighed og et fælles arbejdsgrundlag gennem hele projektforløbet.

Møder

Gruppen afholdt løbende evalueringsmøder gennem projektforløbet. Møderne varierede i form og fokus afhængigt af tidspunktet i ugen og projektets aktuelle status.

Dagligt møde (morgen)

De daglige morgenmøder havde primært fokus på programmets tilstand. Her blev det sikret, at systemet kunne køre i forventet state, og eventuelle pull requests blev gennemgået og løst i fællesskab, hvis det var nødvendigt.

Mandagsmøde (revurdering af ugeplan)

Mandagsmøderne blev anvendt til revurdering af ugeplanen. Her blev den detaljerede plan (jf. Bilag 6: Detaljeret plan) gennemgået og opdateret på baggrund af status fra det foregående fredagsmøde. Formålet var at skabe et fælles overblik over ugens opgaver og justere prioriteringer efter behov.

Fredagsmøde (ugentlig evaluering)

Fredagsmøderne fungerede som ugentlig evaluering af sprintet. Her blev der reflekteret over, om de planlagte opgaver var nået, om GitHub Projects var ajourført, samt om der var behov for arbejde i weekenden. I de tilfælde, hvor der blev arbejdet videre, blev det afklaret, hvem der arbejdede med hvilke opgaver.

Den faste mødestruktur bidrog til en stabil rytme i projektet og gjorde det lettere løbende at følge op på både teknisk status og arbejdsbyrde. Samtidig skabte den klare skelnen mellem planlægning, udførelse og evaluering et fælles grundlag for beslutninger og prioriteringer.

User stories

Arbejdet med at nedbryde user stories i konkrete tasks fungerede overordnet tilfredsstillende. De grundige krav, som blev udledt på baggrund af interviewanalysen, gav et solidt udgangspunkt for formulering af user stories og gjorde det relativt overskueligt at identificere nødvendig funktionalitet. Dette bidrog til, at user stories i høj grad afspejlede reelle behov og forretningslogik.

En udfordring opstod i forhold til timingen af nedbrydningen i tasks. Flere user stories og epics blev tidligt opdelt i mindre opgaver, før gruppen havde opnået fuld forståelse for systemets struktur og afhængigheder. Efterhånden som projektet udviklede sig, blev det derfor nødvendigt at justere eller omskrive enkelte tasks. Med den erfaring taget i betragtning kunne en mere gradvis nedbrydning have været fordelagtig.

Derudover oplevede gruppen en forskel mellem den måde, user stories typisk opdeles på, og den faktiske arbejdsproces i udviklingen. Udviklingsarbejdet tog ofte udgangspunkt i tekniske lag som datamodel, services og integration, mens tasks fra user stories kunne være spredt på tværs af disse lag. Dette gjorde det vanskeligt at tilpasse opgaverne fuldstændigt til arbejdsflowet. Trods dette vurderes størrelsen og omfanget af de fleste tasks som passende, og erfaringen har givet en bedre forståelse for samspillet mellem funktionelle krav og teknisk implementering.

Tidsestimering

Arbejdet med tidsestimering viste en tydelig forskel mellem planlægning på overordnet niveau og estimering af konkrete tasks.

Overordnet tidsestimering – Det store perspektiv

På det overordnede niveau blev tidsplanen overholdt gennem hele projektforsløbet. Gruppen var på intet tidspunkt mere end én planlagt dag bagud, hvilket i høj grad kan tilskrives den detaljerede plan samt den faste mødestruktur. Kombinationen af en klar ugeplan og løbende opfølgning gjorde det muligt tidligt at identificere afvigelser og justere arbejdet, før de udviklede sig til egentlige forsinkelser.

Detaljeret tidsestimering

Den detaljerede tidsestimering af enkelte tasks viste sig derimod at være mere udfordrende. Projektet indeholdt flere teknologier og funktioner, som gruppen ikke tidligere havde arbejdet med, hvilket begrænsede erfaringsgrundlaget for præcise estimater. Dette betød, at visse opgaver tog længere tid end forventet, eksempelvis integrationen af SendGrid. I disse tilfælde blev der løbende justeret i planlægningen ved at omprioritere opgaver og tilpasse backloggen, så fremdriften kunne fastholdes.

Samlet set har erfaringen vist, at detaljeret tidsestimering kræver et erfaringsgrundlag, som først opbygges gennem praktisk arbejde. Den overordnede planlægning fungerede som et stabilt styringsværktøj, mens arbejdet med tasks i højere grad måtte tilpasses undervejs i takt med, at projektets tekniske kompleksitet blev tydeligere.

Rytme i processen

En produktiv arbejdsrytme blev etableret relativt tidligt i projektforløbet. Gruppen havde allerede erfaring med at arbejde sammen fra større projekter på både 1. og 2. semester, hvilket havde skabt fælles forventninger til arbejdsform, kommunikation og struktur. Dette betød, at der fra projektets start var en fælles forståelse for, hvordan arbejdsdagen skulle tilrettelægges for at opnå et effektivt samarbejde.

Et højt fremmøde og dagligt samarbejde på skolen bidrog til et smidigt arbejdsflow, hvor spørgsmål og udfordringer kunne afklares løbende. Den faste mødestruktur, som er beskrevet tidligere i kapitlet, understøttede dette ved at skabe en klar opdeling mellem planlægning, udførelse og evaluering.

Derudover spillede daglige code reviews og brugen af pull requests på GitHub en central rolle. Kode blev som udgangspunkt altid merged via pull requests, selvom gruppens størrelse på to personer kunne have gjort det fristende at vælge en mere uformel tilgang. Ved konsekvent at gennemgå hinandens kode blev der opnået en fælles forståelse for hele kodebasen, hvilket reducerede afhængighed og gjorde det lettere at overtage opgaver ved behov.

Samlet set betød denne fælles tilgang til arbejdsform og samarbejde, at gruppen oplevede at være i en stabil og produktiv rytme allerede fra projektets første fase. Dette dannede et solidt fundament for resten af projektforløbet og understøttede de valg og refleksioner, der er beskrevet i de foregående afsnit.

Bilag

Bilag 1: Citater fra kundebesøg (Video interview med Martin)

Dette bilag indeholder udvalgte og redigerede citater fra et interview med Martin, medarbejder hos Fog. Citaterne danner grundlag for kravene og de forretningsmæssige antagelser, der er beskrevet i rapporten. Udeladelser er markeret med [...]. Mindre sproglige tilretninger er foretaget for at øge læsbarheden (f.eks. fjernelse af fyldord og interjektioner) uden at ændre citatets indhold eller betydning.

“Lige nu kan du bestille en carport. Du kan også gå ind og lave denne forespørgsel til de her specialmål og gør man det, i det øjeblik man trykker på knappen, så lander der en mail herude med de ønsker folk har. Og så har vi vores eget lille program, hvor vi så, lavpraktisk, med de mål folk har tastet ind [...] taster noget bredde og noget længde [...]”
— Martin Kristensen (Fog), interview 00:04:08

“Teknisk set kan du vælge hvad du vil som kunde, og det ville jo være fantastisk hvis [...] alle sammen kunne tilvælges af kunden. Det er der, hvor programmet skal kunne noget mere, og der vil være en masse vedligeholdelse og opdateringer.”
— Martin Kristensen (Fog), interview 00:06:09

“Den har en kort beskrivelse – det er den beskrivelse der bliver skrevet med ud på tegningen, når vi sender kunden. Jamen hvad er det? Det er en carport der er det mål, den har et redskabsrum på der er det mål [...] Det er sådan basis overordnet. Det gør vi selvfølgelig, fordi vi ønsker ikke at dele en stykliste til en kunde der får et tilbud. Han får et tilbud på carporten, og så får han styklisten det øjeblik han køber den.”
— Martin Kristensen (Fog), interview 00:08:13

“Når vi laver de her [carporte med skræddersyede mål], så tager vi jo dialogen med kunden over disken, og så aftaler vi hvad er det der skal ske, hvad er det du godt kan tænke dig.”
— Martin Kristensen (Fog), interview 00:09:45

“Så kommer der faktisk en materialeliste med beskrivelse [...] Den er sat med de længder der skal bruges, for at det her passer med de overlap og den måde det skal laves på.”
— Martin Kristensen (Fog), interview 00:10:26

“Det her er et program fra ’99 [...] Problemet er faktisk, at jeg har et kodeord så jeg kan gå ind og rette priser på det, men ellers kan vi faktisk ikke lave noget. Vi kan ikke rette varer, og vi kan ikke oprette nye varer.”

— Martin Kristensen (Fog), interview 00:13:21

“Det frygtelige er, at vi har en anden udgave liggende [af programmet], som har ligget i syv år, men som aldrig kom til at virke rigtigt. Det den skulle kunne, det var at hente varenumrene direkte nede i vores database. Det vil sige, at når der kom en prisopdatering, så fulgte den med hele vejen.”

— Martin Kristensen (Fog), interview 00:14:05

“Selve programmet er nok til at have med at gøre, men der hvor den bliver tricky, det er det her med [...] den kunne godt tegne det her og visualisere det, men at trække materiale ud af den tegning den genererer...”

— Martin Kristensen (Fog), interview 00:15:15

“Den kan du ikke se herinde, der kan jeg gå herind og arbejde i en prisdatabase.”

— Martin Kristensen (Fog), interview 00:16:06

(Svar på spørgsmål om hvor materialeprisen fremgår.)

“Yes, på mængde og på længde, og så kender den meterpriserne på alle sammen. Og så går den ud og siger jeg skal bruge [...] Der er 10,5 meter og det koster x-antal kroner og så bliver den lagt over i den samlede sum.”

— Martin Kristensen (Fog), interview 00:16:06

(Svar på spørgsmål om prisen er knyttet til de enkelte materialer der fremgår af styklisten.)

“Jeg kan ingenting rette i det andet end jeg kan rette priserne. Det var et rent tilfælde at jeg fandt kodeordet.”

— Martin Kristensen (Fog), interview 00:16:38

(Svar på spørgsmål om hvordan databasen med varekataloget tilgås.)

“En taglægte i dag hedder ikke 38x57, den hedder 38x73. Det kan vi faktisk ikke ændre i vores program. Jeg har ændret prisen til en 73 pris, så maskinen kommer med den rigtige pris. Og når kunden han får den, så får han jo denne her tegning, men styklisten, den er tastet i vores edb-system. Det er efter den vi pakker efter. Så det giver selvfølgelig nogle udfordringer med nogle hjælpe tekster, men det er vi simpelthen nødt til, for materialerne har simpelthen ændret sig på 20 år.”

— Martin Kristensen (Fog), interview 00:17:13

“Vi er en ganske almindelig trælast. Alt hvad vi sælger herfra, det er varer taget ude fra hylden. [...] Så ved at der blev leveret en stykliste, så ville vores kompetence være væk og så ville der være fri konkurrence. [...] Det er jo en af vores kompetencer. Det har vi valgt at satse på for længe siden at vi kan, og det vil vi selvfølgelig også have nogle penge for.”

— Martin Kristensen (Fog), interview 00:19:30

(Svar på spørgsmål om hvorfor styklisten ikke bliver sendt til kunden med et tilbud.)

“Vi kan egentlig godt lide at der er en sælger indover. Der gør, så har han lavet den personligt. Det vil sige at han kan følge op [...] Så har man denne her personlige tilgang. Hvor hvis folk selv klikker sig ind og printer sin egen tegning og går ud igen, Så har du ikke den samme.”

— Martin Kristensen (Fog), interview 00:21:20

“Begge dele. Det kan man ikke sige.”

— Martin Kristensen (Fog), interview 00:25:11

(Svar på spørgsmål om hvorvidt antallet af stolper afhænger af afstand eller antal.)

“Det kan man ikke sige. Det kommer an på vægt og belastning og så videre. Der er ikke nogen entydig regel for det. Der er du nødt til at tage bredden på taget, hvad det er for en tagtype, altså hvad vægt den når op på pr. m², og så regne trykket ud på, hvor meget den trykker på remmen.”

— Martin Kristensen (Fog), interview 00:25:20

(Svar på spørgsmål om hvor langt der må være mellem stolper i en carport.)

Bilag 2: Risikoanalyse

ID	Risiko	Alvor	Sandsynlighed	Risikoniveau	Plan for forebyggelse / afværgning
1	Konkurs ved kunden	Uacceptabelt	Usandsynlig	Ekstrem	Product owner sørger for at lave en aftale om ratebetaling forud for projektet. Stands alt udvikling øjeblikkeligt ved konkurs
2	Projektet lever ikke op til kundekravene	Uacceptabelt	Muligt	Ekstrem	Product owner sørger for løbende kontakt mellem denne og stakeholderen ved behov og involverer som minimum denne mellem hvert sprint
3	Betalingsstop fra kunden	Uacceptabelt	Muligt	Ekstrem	Product owner kontakter kunden. Stands alt udvikling øjeblikkeligt
4	Mergekonflikt i main branch	Uønsket	Muligt	Ekstrem	Tech Lead har ansvaret for at reviewe kode samt test forud for merge til main. Det er som udgangspunkt kun Tech lead der kan merge med main - alle andre tilfælde skal aftales med Tech lead eller ved fravær med Team lead
5	Ændringer i firmastruktur ved stakeholderen	Uønsket	Muligt	Ekstrem	Product owner etablerer kontakt til stakeholderen / ny ledelse of bed om en klar formulering af hvilke konsekvenser ændringerne har for projektet
6	Overskridelse af budget	Uønsket	Muligt	Ekstrem	Team lead har ansvaret for at styre arbejdstid blandt udviklerteamet. Der er ikke plads i budgettet til merarbejde. Såfremt en udvikler ikke kan nå de aftalte tasks kontaktes Scrum master øjeblikkeligt. Scrum master revurderer tasks og uddelegerer om nødvendigt

ID	Risiko	Alvor	Sandsynlighed	Risikoniveau	Plan for forebyggelse / afværgning
7	Servervedbrud ved host	Tolereres	Sandsynligt	Høj	Product owner tager i samarbejde med Tech lead kontakt til Stakeholderen for at udbedre dette. Såfremt at servernedbruddet er længerevarende (over 24 timer) skal Scrum masteren lave en Kanban / Scrum-board med tasks der kan løses uafhængigt af server (lokalt). Dette videregives til Udviklingsteamet hurtigst muligt. Udviklingsteamet går selvstændigt færdiggøre igangværende tasks såfremt muligt - ellers søger disse at trække en opgave der kan løses lokalt indtil Scrum masteren fremlægger det nye Kanban / Scrum-board
8	Mergekonflikt i developer branch	Tolereres	Sandsynligt	Høj	Der må udelukkende merges i forbindelse med pull requests (med undtagelse af Tech lead). Såfremt der opstår mergekonflikt løses dette optimalt i featurebranches via Git Pull forud for merge til dev. Ved tvivl, kontakt Tech lead
9	Fravær ved stakeholderen	Tolereres	Sandsynligt	Høj	Product owner har ansvaret for at etablere et tydeligt hierarki ved stakeholderen/kunden - Fastslå hvem der bestemmer ved fravær
10	Nedbrud ved eksterne systemer	Tolereres	Sandsynligt	Høj	Såfremt at nedbruddet er længerevarende (over 24 timer) skal Scrum masteren lave en Kanban / Scrum-board med tasks der kan løses uafhængigt af gældende system (lokalt). Dette videregives til Udviklingsteamet hurtigst muligt. Udviklingsteamet går selvstændigt færdiggøre igangværende tasks såfremt muligt - ellers søger disse at trække en opgave der kan løses lokalt indtil Scrum masteren fremlægger det nye Kanban / Scrum-board

ID	Risiko	Alvor	Sandsynlighed	Risikoniveau	Plan for forebyggelse / afværgning
11	Kundekrav ændres	Acceptabel	Sandsynligt	Høj	Product Owner informerer hurtigst muligt udviklingsteamet om relevante ændringer. Scrum master opdaterer øjeblikkeligt Kanban / Scrum-board med ændringer
12	Tab af lokale filer / data (mangel på backup / cloud)	Uacceptabelt	Muligt	Høj	Tech lead laver forud for projektet en codingstandards.md hvor der sættes et krav om hyppigheden af commits. Udviklingsteamet skal alle briefes forud for projektets start og Tech lead skal løbende kontrollere at dette bliver overholdt
13	Beregningsalg oritme fungerer ikke efter hensigten	Uacceptabelt	Muligt	Høj	Udviklingsteamet: Sørg for at anvende beregninger der er i overensstemmelse med dem der anvendes af kunden. Tech lead har ansvaret for at beregningsalgoritmerne er testet forud for at Product Owner fremviser disse til stakeholderen
14	Mismatch i software versionering i udviklerteamet	Uønsket	Muligt	Høj	Tech lead laver forud for projektet en codingstandards.md hvor der informeres om versionering. Udviklingsteamet skal alle briefes forud for projektets start og Tech lead skal løbende kontrollere at dette bliver overholdt samt løbende følge op ved ændringer
15	Ufuldkommen analyse af projekt	Uønsket	Muligt	Høj	Udviklerteamet: Identificeres mangel på analyse eller fejlanalyse meldes dette videre til Scrum master der revurderer projektets forløb
16	Mangel på struktur i projektet	Uønsket	Muligt	Høj	Scrum master bekendtgør udviklerteamet med Kanban / Scrum-board for projektet. Scrum master opdaterer dette løbende og denne har ansvaret for løbende at afholde Scrum møder

ID	Risiko	Alvor	Sandsynlighed	Risikoniveau	Plan for forebyggelse / afværgning
17	Misforståelse af kundekrav i udviklings-teamet	Tolereres	Muligt	Medium	Scrum master sørger for daglige Scrum møder. Udviklingsteamet gennemgår de daglige tasks med Product Owner ved tvivl. Såfremt at Product Owner støder på en tydelig misforståelse ved teamet skal denne kontakte relevant personel for at udbedre
18	Intern modstand fra brugere / gidsler	Tolereres	Sandsynligt	Medium	Tech lead udfærdiger en brugermanual til softwaren - denne skal henvendes til brugeren og må ikke være "for teknisk". Product owner tilbyder stakeholderen hjælp med at forklare kunden (Gidslerne) om fordelene ved den nye software
19	Sygdom i udviklings-teamet	Tolereres	Sandsynligt	Medium	Hver udvikler skal have en klar plan for overdragelse og status på igangværende opgave i tilfælde af fravær. Scrum master har ansvaret for at uddelegere opgaver i forbindelse med dette
20	IT-nedbrud i udviklings-teamet	Acceptabel	Muligt	Medium	Ved nedbrud på personligt IT-udstyr kontaktes IT-support øjeblikkeligt. Såfremt disse ikke er til stede, kontaktes Tech lead der råder over en reserve IT-løsning. Tech lead har ansvaret for at prioritere hvilken udvikler der har adgang til reserve IT-løsningen såfremt at flere udviklere har behov. IT-reserve løsningen må ikke være i "fast udlån" - IT-support skal afhjælpe IT problemer indenfor maksimalt 5 arbejdsdage. Er dette ikke muligt skal IT support kontakte Team lead der her har ansvaret for at træffe en endelig beslutning

ID	Risiko	Alvor	Sandsynlighed	Risikoniveau	Plan for forebyggelse / afværgning
21	Overskridelse af tidsplan i udviklings-teamet	Tolereres	Sandsynligt	Medium	Udviklingsteamet: Meld frem til Scrum master og send her en ny estimeret tid for færdiggørelse. Scrum master: Ved gentagende overskridelser - revurder estimering af tid til tasks. Ved næste daglige Scrum møde revurderes prioritering af opgaver. Såfremt overskridelsen er kritisk, har Scrum master ansvaret for at vurdere og prioritere opgaver og derefter informere relevant personel
22	Inflation (byggematerialer / lønninger)	Acceptabel	Muligt	Lav	Team lead orienter udviklingsteamet omkring inflationens konsekvenser for projektet. Product owner tag kontakt til kunden om nødvendigt

Bilag 3: Domain Model Script - Relationer

```
Account <|- Customer
Account <|- Seller

Customer "1" -- "0..*" Inquiry : places
Inquiry "1" -- "1" Carport : contains
Inquiry "1" -- "0..1" Offer : generates
Seller "1" -- "0..*" Offer : handles/edits
Seller "1" -- "0..*" MaterialCatalog : manages

Customer "1..*" -- "0..*" Notification : receives
Inquiry "1" -- "1" Notification : triggers
Offer "1" -- "1" Notification : triggers
Order "1" -- "1" Notification : triggers

Offer "0..*" -- "1" Customer : sent to
Offer "1" -- "0..1" Order : becomes (if accepted)
Offer "1" -- "1" Price
Offer "1" -- "1" CarportDrawing

Order "0..*" -- "1" Customer : belongs to
Order "1" -- "1" BillOfMaterials : includes

Carport "1" -- "1" Measurements
Carport "1" -- "0..1" Shed
Carport "0..*" -- "*" Material : uses
Shed "0..1" -- "1" Measurements
Shed "0..1" -- "*" Material : uses
Material "1" -- "1" MaterialCategory : is type of
Material "0..*" -- "1" MaterialCatalog : contains

BillOfMaterials "1" -- "1..*" Material : contains
```

Bilag 4: User stories

Dette bilag indeholder samtlige user stories, som er udarbejdet i forbindelse med projektet. User stories er formuleret på baggrund af de krav, der er beskrevet tidligere i rapporten, og repræsenterer gruppens fortolkning og omsætning af disse krav til funktionelle behov i systemet. User stories er udarbejdet som en del af opgaven og er ikke formuleret i samarbejde med kunden.

Enkelte user stories er markeret som EPIC og dækker større funktionelle områder, som er opdelt i underliggende tasks.

Alle user stories er estimeret efter sværhedsgrad, S (small) < M (medium) < L (large), ud fra en vurdering af kompleksitet, da gruppen ikke havde et tilstrækkeligt grundlag for at estimere i tid.

US-1

Rolle: Kunde

Som kunde ønsker jeg at oprette en konto hos fog, så jeg kan gemme mine oplysninger og bestille en skræddersyet carport.

Acceptkriterier:

Givet at jeg besøger hjemmesiden, når jeg vælger opret ny bruger, udfylder registreringsformularen og klikker "Opret konto", så oprettes min konto og jeg kan logge på.

Estimat: M

US-2

Rolle: Kunde

Som kunde ønsker jeg at logge ind med min e-mail og adgangskode, så jeg kan få adgang til min igangværende forespørgsler og ordrer.

Acceptkriterier:

Givet at jeg har en eksisterende konto, når jeg indtaster en korrekt e-mail og adgangskode så får jeg adgang til min personlige kundeside, hvor jeg kan se mine forespørgsler, tilbud og betalte ordre.

Estimat: S

US-3 EPIC

Rolle: Kunde

Som kunde ønsker jeg at kunne angive mål for carporten (længde og bredde), hældning på tag og tilvalg om skur ønskes, så jeg kan skræddersy en carport efter mine ønsker.

Estimat: M

US-3.1 — Indtast mål**Rolle:** Kunde

Som kunde ønsker jeg at indtaste mål (længde og bredde) på carporten, så konstruktionen passer til mine behov.

Acceptkriterier:

Givet at jeg er på "Byg din carport formular"-siden, når jeg vælger længde, bredde i cm så vil værdierne gemmes til brug i beregningen af tilbuddet.

Estimat: S

US-3.2 — Vælg tagtype og hældning**Rolle:** Kunde

Som kunde ønsker jeg at vælge tagtype og hældning, så jeg kan få den tagløsning, jeg foretrækker.

Acceptkriterier:

Givet at jeg er på "Byg din carport formular"-siden når jeg vælger en tagtype (fx fladt tag, sadeltag, plasttag osv.) Og indstiller hældningen, så gemmes valgene og bruges i beregningen af tilbuddet.

Estimat: M

US-3.3 — Tilvalg af skur**Rolle:** Kunde

Som kunde ønsker jeg at kunne vælge et skur samt angive skurets mål, så jeg kan få et skræddersyet skur i den størrelse, som opfylder mine behov.

Acceptkriterier:

Givet at jeg er på "Byg din carport formular"-siden, når jeg vælger "Tilføj skur" og indtaster skurets længde og bredde, så gemmes disse oplysninger bruges i beregningen af tilbuddet.

Estimat: M

US-4**Rolle:** Kunde

Som kunde ønsker jeg at oprette en forespørgsel på en skræddersyet carport, så en sælger kan udarbejde et tilbud til mig.

Acceptkriterier:

Givet at jeg er logget ind, når jeg udfylder og sender carportformularen, så gemmes min forespørgsel i systemet. Sælger kan derefter se at der er en ny forespørgsel i systemet.

Estimat: M

US-5**Rolle:** Kunde

Som kunde ønsker jeg at modtage en e-mail, når der er oprettet et tilbud på min carport forespørgsel.

Acceptkriterier:

Givet at en sælger har oprettet et tilbud, når systemet registrerer tilbuddet, så modtager jeg en e-mail med besked om, at der er et nyt tilbud på min kundeside.

Estimat: M

US-6**Rolle:** Kunde

Som kunde ønsker jeg at kunne se mit tilbud på "Min side" under tilbud, når jeg har modtaget et nyt tilbud, så jeg kan se tilbuddet og en tegning af carporten.

Acceptkriterier:

Givet at en sælger har oprettet et tilbud til mig, når jeg logger ind eller er logget ind, og at jeg er på "Min side" når jeg åbner tilbuddet, kan jeg se en kort beskrivelse af carport, tegning og pris.

Estimat: L

US-7**Rolle:** Kunde

Som kunde ønsker jeg at acceptere et tilbud, så jeg kan få styklisten og min ønskede carport.

Acceptkriterier:

Givet at jeg har modtaget et tilbud, når jeg klikker "Accepter" og gennemfører betalingen. Så registreres betalingen, og jeg kan se styklisten og modtage min carport.

Estimat: L

US-8**Rolle:** Kunde

Som kunde ønsker jeg at modtage en e-mail når jeg har afsendt forespørgsel på carport, så jeg ved at min forespørgsel er registret.

Acceptkriterier:

Givet at jeg er åbner min personlige mail, når webapplikationer modtager min informationer, så modtager jeg en e-mail om at min forespørgsel er registret og vil blive behandlet hurtigst mulig.

Estimat: S

US-9**Rolle:** Kunde

Som kunde ønsker jeg at kunne afvise et tilbud, hvis tilbuddet ikke lever op til mine forventninger.

Acceptkriterier:

Givet at jeg har modtaget et tilbud under tilbud på "Min side" og åbnet det, når jeg klikker afvis tilbud, så markeres tilbuddet som afvist.

Estimat: S

US-10**Rolle:** Kunde

Som kunde ønsker jeg at kunne se mine betalte ordrer med tilhørende stykliste, tegning og pris, så jeg nemt kan få overblik over mine købte carporte.

Acceptkriterier:

Givet at jeg er logget ind, Når jeg åbner "Min side" og vælger en ordre fra ordrelisten, så vises en oversigt over mine tidligere betalte ordrer.

Når jeg klikker på en specifik ordre, så vises ordredetaljer, stykliste, tegning og samlet pris.

Estimat: M

US-11**Rolle:** Sælger

Som sælger ønsker jeg at logge ind på webapplikationen, så jeg kan bruge den til at servicere mine kunder.

Acceptkriterier:

Givet at jeg har et gyldigt login, når jeg indtaster mine oplysninger. Så får jeg adgang til sælgersiden, hvor jeg kan se "Forespørgsler", "Tilbud", "Materialer" og "Ordre".

Estimat: S

US-12**Rolle:** Sælger

Som sælger ønsker jeg at se, hvem der er logget ind (e-mail og ID), så jeg kan sikre, at jeg er korrekt logget på og ikke bruger en andens konto.

Acceptkriterier:

Givet at en sælger er logget ind, når jeg ser brugeroplysninger øverst ved siden af profil ikon, så vises e-mail og ID på den aktuelle bruger.

Estimat: S

US-13**Rolle:** Sælger

Som sælger ønsker jeg at se alle nye forespørgsler fra kunder, så jeg hurtigt kan reagere og godkende et tilbud.

Acceptkriterier:

Givet jeg er logget ind, når jeg åbner "Forespørgsler", så vises en liste over alle nye kundeforespørgsler med kundeoplysninger og dato for forespørgslen.

Estimat: M

US-14**Rolle:** Sælger

Som sælger ønsker jeg at se alle mine tilbud, så jeg kan følge deres status.

Acceptkriterier:

Givet at jeg er logget ind, når jeg åbner "Tilbud", så vises en oversigt med status for hvert tilbud.

Estimat: S

US-15**Rolle:** Sælger

Som sælger ønsker jeg at se alle mine ordre, så jeg kan nemt finde en tidligere ordre.

Acceptkriterier:

Givet at jeg er logget ind, når jeg åbner "Ordre", så vises en liste over alle ordrer med relevante detaljer (kunde, dato, beløb, status).

Estimat: M

US-16**Rolle:** Sælger

Som sælger ønsker jeg at redigere webapplikationen forslag til stykliste, så jeg kan kontrollere og tilpasse konstruktionen for nøjagtighed.

Acceptkriterier:

Givet at jeg arbejder på et tilbud, når jeg ændrer materiale-mængder, så opdateres stykliste og totalpris automatisk.

Estimat: M

US-17**Rolle:** Sælger

Som sælger ønsker jeg at åbne kundens forespørgsel som carport-tegning, så jeg kan gennemgå designet i detaljer.

Acceptkriterier:

Givet at Sælger er logget ind og er ved at se et tilbud eller ordre detalje igennem, er der tilknyttet en tegning, når jeg klikker "Se tegning", så åbnes tegningen i en ny side.

Estimat: L

US-18**Rolle:** Sælger

Som sælger ønsker jeg at justere dækningsgraden på tilbuddet, så jeg kan optimere chancen for, at kunden accepterer mit tilbud.

Acceptkriterier:

Givet at jeg redigerer et tilbud, når jeg ændrer totalprisen og gemmer, så opdateres totalprisen i systemet, hvor jeg stadig kan se kostprisen, dækningsgrad, så jeg kan opveje konsekvenserne, før jeg sender tilbuddet.

Estimat: M

US-19**Rolle:** Sælger

Som sælger ønsker jeg at et tilbud har en udløbsdato, så kunden ved hvor længe tilbuddet er gyldigt.

Acceptkriterier:

Givet jeg opretter et tilbud, når tilbuddet afsendes, så er tilbuddet aktivt i 14 dage, så vises udløbsdatoen for kunden og tilbuddet kan ikke accepteres efter denne dato.

Estimat: S

US-20**Rolle:** Sælger

Som sælger ønsker jeg at slette et tilbud, så forældede eller annullerede tilbud fjernes fra systemet.

Acceptkriterier:

Givet at et tilbud er afvist eller udløbet og jeg er på siden "Tilbud", når jeg trykker "Slet" ud foran et tilbud, så fjernes tilbuddet permanent fra systemet.

Estimat: S

US-21**Rolle:** Sælger

Som sælger ønsker jeg at se, hvornår en forespørgsel blev oprettet, så jeg kan vurdere, hvor længe kunden har ventet på svar.

Acceptkriterier:

Givet at jeg åbner "Forespørgsler", når jeg kigger på listen over forespørgsler, så kan jeg se hvor lang tid siden en forespørgsel er oprettet og givet jeg åbner en specifik forespørgsel åbnes, når detaljevisningen er indlæst, så vises dato og klokkeslæt for oprettelsen.

Estimat: S

US-22**Rolle:** Sælger

Som sælger ønsker jeg at opdatere materialer i databasen, så priser og beskrivelser altid er korrekte.

Acceptkriterier:

Givet at jeg har adgang til materialebasen, når jeg redigerer et materiale og klikker "Opdater", så opdateres oplysningerne i webapplikationen.

Estimat: M

US-23**Rolle:** Sælger

Som sælger ønsker jeg at tilføje nye materialer til databasen, så kunderne får adgang til de nyeste produkter som virksomheden har at tilbyde.

Acceptkriterier:

Givet at jeg er logget ind som sælger og står på siden "Materialer", når jeg indtaster oplysninger om et nyt materiale og klikker "Gem", så tilføjes det til systemets materialeliste.

Estimat: M

US-24**Rolle:** Sælger

Som sælger ønsker jeg at fjerne materialer i databasen, så materialer der ikke er tilgængelige længere, kan blive fjernet.

Acceptkriterier:

Givet at jeg er logget ind som sælger og står på siden "Materialer", når man står ud foran et materiale og trykker på "Slet", sletter jeg materialet, så bliver materialet fjernet fra databasen.

Estimat: S

US-25**Rolle:** Kunde eller sælger

Som kunde eller sælger ønsker jeg at kunne logget ud af webapplikationen, så mine oplysninger er sikre når jeg forlader computeren.

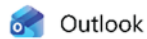
Acceptkriterier:

Givet at jeg er logget ind, når jeg trykker "Log ud", så bliver jeg logget ud og returneret til forsiden.

Estimat: S

Bilag 5: Kunde e-mails

Bekræftelse på forespørgsel



Bekræftelse af forespørgsel - Fog Carporte

Fra Johannes Fog Byggemarked <[redacted]>

Dato Man 15-12-2025 09:56

Til [redacted]



Kære Morten Jensen

Tak for din forespørgsel på en carport. Vi har modtaget dine indsendte mål og oplysninger.

En af vores medarbejdere kontakter dig hurtigst muligt for at gennemgå dine ønsker og sikre, at alt stemmer, så vi kan tilbyde den bedst mulige løsning.

Venlig hilsen

Fog



Johannes Fog A/S
Firskovvej 202800
Lyngby
CVR-nr. 16314439

Tilbud



Dit tilbud er klar på johannesfog.dk

Fra Johannes Fog Byggemarked <[redacted]>

Dato Man 15-12-2025 09:58

Til [redacted]



Kære Morten Jensen,

Dit tilbud er nu klar!

Log ind på din Fog-konto for at se dit tilbud og de informationer, vi har samlet til dig.

[Se dit tilbud her](#)

Har du spørgsmål eller behov for justeringer, står vores medarbejdere klar til at hjælpe dig videre.

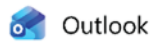
Venlig hilsen

Fog



Johannes Fog A/S
Firskovvej 202800
Lyngby
CVR-nr. 16314439

Ordrebekræftelse



Ordre bekræftelse - Fog Carporte

Fra Johannes Fog Byggemarked <[redacted]>

Dato Man 15-12-2025 09:59

Til [redacted]



Kære Morten Jensen,

Tak for din bestilling hos Fog!

Ordre detaljer:

Ordrenummer: 32

Ordredato: 15. december 2025

Ordrestatus: Betalt

Vi har nu modtaget din ordre, og den er registreret i vores system. Vi takker for din tillid og ser frem til at levere dit projekt.

Herunder finder du en oversigt over det, du har bestilt:

Detaljer om din carport:

Længde: 780 cm

Bredde: 600 cm

Højde: 230 cm

Tagtype: Trapez tag

Prisoversigt:

Pris uden moms: 33.600,00 kr.

Totalpris: 42.000,00 kr.

Du kan se hele din ordre med styklister, tegninger og alle detaljer direkte på din Fog-konto ved at klikke her:

[Se din fulde ordre](#)

Kontakinformation:

Hvis du har spørgsmål til ordren, ændringer eller ønsker til projektet, er du velkommen til at kontakte din personlige sælger:

Kontaktperson: Martin [REDACTED]

Direkte tlf.: [REDACTED]

E-mail: [REDACTED]

Venlig hilsen

Fog



Johannes Fog A/S
Firskovvej 20
2800 Lyngby
CVR-nr. 16314439

Bilag 6: Detaljeret plan

Uge: 47				
Scrum master: Daniel		Product owner: Daniel		Tech lead: Morten
Mandag	Tirsdag	Onsdag	Torsdag	Fredag
ANALYSE • TO-BE diagram • Domain model • Risiko analyse • Mock up (Figma)	ANALYSE • Class diagram • ERD • Codingstandard.txt	KODE START • .gitignore • Projekt struktur oprettes • Planlæg arbejdsfordeling	DATABASE • Mappers (TDD) • SQL-script	DATABASE • Mappers (TDD) • Indsæt test-data • Afrund ugen
Uge: 48				
Scrum master: Morten		Product owner: Morten		Tech lead: Daniel
Mandag	Tirsdag	Onsdag	Torsdag	Fredag
• Revurder ugen • Vejledning 09:30 • Færdiggør Mappers • Start servicelag	FORRETNINGS-LOGIK: BEREGER SVG	FORRETNINGS-LOGIK	FORRETNINGS-LOGIK	FÆRDIGGØR FORRETNINGS-LOGIK • Afrund ugen
Uge: 49				
Scrum master: Daniel		Product owner: Daniel		Tech lead: Morten
Mandag	Tirsdag	Onsdag	Torsdag	Fredag
• Revurder ugen • Vejledning 09:30 • Færdiggør svg	SENDGRID CONTROLLER FRONT END	CONTROLLER FRONT END	CONTROLLER FRONT END	CONTROLLER FRONT END • Afrund ugen

Uge: 50				
Scrum master: Morten		Product owner: Morten		Tech lead: Daniel
Mandag	Tirsdag	Onsdag	Torsdag	Fredag
RAPPORT START • Revurder ugen • Vejledning 09:30 • Klargør materialer til bilag • Klargør .config fil til styrende variabler	RAPPORT • Opsæt rapport skabelon • Sorter noter til rapport	RAPPORT • Afrund opgaver på Github Projects • Sorter noter til rapport	RAPPORT • Fejlhåndtering: Løs kritiske, fejl opdaget • Fejlhåndtering: Løs kritiske ikke-færdige opgaver fra Github Projects	RAPPORT • Afrund ugen • Status på rapport
Uge: 51				
Scrum master: Daniel		Product owner: Daniel		Tech lead: Morten
Mandag	Tirsdag	Onsdag	Torsdag	Fredag
STATUS PÅ PROJEKT • Vejledning 09:30 • Finish • Upload på GitHub • Kontrol	GENNEMLÆSNING DEPLOYMENT • Code freeze • Merge til main • Upload på GitHub	GENNEMLÆSNING VIDEO DEMO README • Aflever hvis muligt	AFLEVER PROJEKT !	